

An Approach to Ensuring Consistency in Peer-to-Peer Real-Time Group Editors

DU LI & RUI LI

Department of Computer Science, Texas A&M University, College Station, Texas, 77843-3112, USA (E-mail: lidu@cs.tamu.edu)

Abstract. Real-time group editors allow distributed users to edit a shared document at the same time over a computer network. Operational transformation (OT) is a well accepted consistency control method in state-of-the-art group editors. Significant progress has been made in this field but there are still many open issues and research opportunities. In particular, established theoretic OT frameworks all require that OT algorithms be able to converge along arbitrary transformation paths. This property is desirable because group editors that implement such algorithms will not rely on a central component for achieving convergence. However, this has not been achieved in any published work to our knowledge. We analyze the root of this problem and propose a novel state difference based transformation (SDT) approach which ensures convergence in the presence of arbitrary transformation paths. Our approach is based on a novel consistency model that is more explicitly formulated than previously established models for proving correctness. SDT is the first and the only OT algorithm proved to converge in peer-to-peer group editors.

Key words: computer-supported cooperative work, consistency maintenance, group editing, groupware, operational transformation

1. Introduction

Real-time group editors (e.g., Ellis and Gibbs, 1989; Bellini et al., 2002; Xia et al., 2004) allow a distributed group of users to edit the same document collaboratively at the same time over a computer network, e.g., the Internet. They have been used frequently by numerous groupware researchers, to name but a few, (Ellis and Gibbs, 1989; Hymes and Olson, 1992; Dewan et al., 1994; Knister and Prakash, 1994; Neuwirth et al., 1994; Sun et al., 1998), as an effective research vehicle and model of a wide range of distributed interactive groupware applications that feature coordinated manipulation of shared data objects.

Over the past decade operational transformation (OT) (Ellis and Gibbs, 1989; Sun and Ellis, 1998; Davis et al., 2002) has become an established method for consistency maintenance in group editors. Compared to alternative concurrency control methods such as locking and serialization, OT has been found uniquely promising in achieving convergence, causality and intention preservation, while achieving high responsiveness and concurrent work at the same time (Sun et al., 1998).

The main property of OT, allowing users to edit any part of the shared data at any time, in particular matches the vision of Bentley and Dourish (1995b) of developing collaborative systems as a customizable collaboration medium. In light of their arguments, group editors with this property do not impose any constraint on how people use the technology. Users are allowed to use group editors for either synchronous or asynchronous editing. With the “right” single-user features (e.g., formatting) and multi-user features (e.g., Sun, 2002a; b), group editors appear capable of accommodating a variety of editing purposes, work styles, and processes (Baecker et al., 1993; Noel and Robert, 2004).

In exploration of the new opportunities brought in by OT to pursue these promises, perseverant efforts have been made since the seminal work of Ellis and Gibbs (1989). These have resulted in significant progress in the field, e.g., the distinction between transformation functions for different purposes (Suleiman et al., 1998; Sun et al., 1998), the solution of various consistency problems (Ressel et al., 1996; Sun et al., 1998; Sun, 2002b), the implementation of group undo (Knister and Prakash, 1994; Sun, 2002b), the generalization to string, sequence, and tree based operations and data structures (Sun et al., 1998; Bellini et al., 2002; Davis et al., 2002), and the transparent leveraging of full-fledged single-user editors and word processors (Li and Li, 2002; Xia et al., 2004). Despite these achievements, there are still many open issues and research opportunities, such as the maintenance of user intentions and semantic consistency (Sun and Ellis, 1998).

1.1. CONSISTENCY PROBLEMS

This article primarily aims to address the following two problems: The first is divergence. Current theoretic OT frameworks as established by Ressel et al. (1996), Sun et al. (1998) all require the satisfaction of two special conditions (TP1 and TP2 (Ressel et al., 1996)) such that the OT algorithm can converge along arbitrary transformation paths. This requirement poses significant challenges in the design of OT algorithms. However, it greatly simplifies the design of group editors, since group editors based on such OT algorithms will be able to ensure convergence in any communication architecture, with or without a central component. Unfortunately, this has turned out extremely difficult to achieve. While GOT (Sun et al., 1998) and TIBOT (Li et al., 2004) are fully replicated, NICE (Shen and Sun, 2002) and SOCT3/4 (Vidot et al., 2000) are centralized, they all achieve convergence by enforcing a unique transformation path at all sites in their control algorithms. A more general solution to this problem which ensures convergence in the presence of arbitrary transformation paths still seems worth research.

The second problem involves the preservation of intentions. OT has been found promising in preserving intentions (Sun et al., 1998) and the CCI

consistency model of Sun et al. (1998), namely, convergence, causality and intention preservation, has been widely accepted in interactive groupware applications, e.g., (Suleiman et al., 1998; Bellini et al., 2002; Sun and Chen, 2002; Sun, 2002a, b). However, there are scenarios which are identified as intention-preserving in Sun et al. (1998) but obviously violate operation intentions. The root of this problem lies in our generally inadequate understanding of user intentions and the difficulties of knowledge formalization. The lack of provable correctness conditions in previous work renders it difficult to verify the correctness of OT algorithms.

1.2. TECHNICAL APPROACH AND CONTRIBUTIONS

To systematically address these two consistency problems, we propose a novel consistency model called CSM and a novel state difference based transformation algorithm called SDT. Our work reveals a new strategy in consistency maintenance in group editors by explicitly formulating verifiable system-level correctness conditions. As a result, the correctness of OT algorithms can be proved for the first time, to our best knowledge. In particular, SDT is the first (and the only so far) OT work that is proved to converge along arbitrary transformation paths.

Our notion of CSM consistency, namely, causality, single-operation effects and multi-operation effects relation preservation, is a pure system-level consistency model. Due to limitations of current technologies, realistically there is no easy way, and sometimes it is harmful, for us to formalize exactly what the user intentions are in collaborative systems (Grudin, 1994; Shipman and Marshall, 1999). Hence we focus our attention to maintaining *syntactic consistency*, the conditions of which can be rigorously formalized and verified. Interactive groupware applications that are CSM-consistent can serve well as a basis for constructing a reliable collaboration medium, above which high-level user interfaces, e.g., mechanisms for awareness (Gutwin and Greenberg, 2002) and error recovery (Sun, 2002b), can be provided to help end users preserve their actual intentions and maintain *semantic consistency*. This separation of concerns in consistency maintenance is significant in groupware engineering: while syntactic consistency can be formally proved, semantic consistency and intention preservation are left to the interpretation of human users and evaluated in usability experiments.

Our SDT algorithm, on the other hand, is able to ensure CSM consistency at the system level. In particular, it introduces a concept of operation effects relation and an approach to capture the correct effects relation between any operations. Our new transformation functions reinterpret OT based on this notion. Aided by a novel control algorithm, our approach is able to maintain the correct operation effects relation. As a result, convergence is automatically achieved along arbitrary transformation paths, because all sites have the

same set of objects (or operation effects) that are in the same total order. Our transformation functions introduce extra parameters for restoring operation effects relation. Our approach provides an alternative solution to the convergence problem that is able to work in both systems that are fully replicated and systems that have central components. Our approach is formally proved to achieve convergence along arbitrary transformation paths and ensure CSM consistency.

1.3. ORGANIZATION OF ARTICLE

The rest of the article is organized as follows. Section 2 builds a background of group editors and OT. Section 3 illustrates the inconsistency problem and analyzes the root of the problem. Section 4 introduces our CSM consistency model and compares it with the well-established CCI model. Sections 5, 6, and 7 reinterpret OT with the notion of operation effects relation and elaborate how to compute operation effects relation. The operation integration algorithm is given in Section 8. A complete example is given in Section 9 to illustrate the main points of our approach. This is followed by a correctness proof of our approach in Section 10. Related work is compared in Section 11. Finally Section 12 concludes this article.

2. Background

Group editors are an interesting research vehicle because they are able to model a wide range of collaborative applications that feature coordinated manipulation of shared data (Dewan et al., 1994). OT algorithms have been well accepted for consistency maintenance in group editors because of their interactive properties: high responsiveness and high concurrency. In this section we give a summary of group editors and OT.

2.1. THE GROUP EDITOR MODEL AND NOTATIONS

As a convention in group editors (Sun and Ellis, 1998), we model the shared document as a linear string and focus on two primitive characterwise operations, insert and delete. Recent work (Sun et al., 1998; Bellini et al., 2002; Davis et al., 2002) has generalized this classic model to more sophisticated data structures and operations with only minor algorithmic extensions. Our use of the classic model will help focus on analyzing and solving the real problems.

In this model, the position of the first character in the string is zero. Significant editing operations are normalized as *insert*(S, P) and *delete*(S, P), which insert and delete string S at position P of the document, respectively. Given an operation O , we use function $T(O)$ to denote its type (INS or DEL),

$P(O)$ for its position parameter, and $L(O)$ the length of string S . In principle, S can be a string (word, sentence, paragraph, section, etc.) of any length (Sun et al., 1998). To simplify discussions, however, we are only interested in characterwise operations. Hence $L(O)=1$ for any O unless stated or implied otherwise. Function $C(O)$ (or $S(O)$) represents the character (or string) to be inserted or deleted by O .

Notation $S + O = S'$ means the execution of O on state S results in state S' . If $S + O_i = S'$ and O_j is executed on state S' , we say that operation O_i contextually precedes O_j and these two operations are contextually serialized. Notation $[O_1, \dots, O_n]$ represents a (contextually serialized) operation sequence, meaning that for any $i, 1 \leq i \leq n-1, O_i$ contextually precedes O_{i+1} . Notation $\{O_1, \dots, O_n\}$ represents a set of operations, their order of appearance not implying contextual serialization. Notation $S + X = S'$ means that the execution of X on state S results in state S' , where X can be a single operation, a sequence, or a set of operations.

Let SQ be a sequence, notation $SQ[i]$ denotes its i th element. $SQ[i, j]$ denotes the subsequence of SQ which inclusively ranges from its i th element to the j th element and $i \leq j$. The subsequence is empty if $i > j$. Infix operator '+', as used in $SQ_1 + SQ_2$, returns the concatenation of its two operand sequences.

The shared document is replicated at each site. We assume that each site is only allowed to generate editing operations sequentially on the local replica. The following definitions of generation, execution, and definition states are similar to those in Sun et al. (1998). Operations are first executed locally and then executed at remote sites. The document state on which an operation O is generated, the *generation state* of O , may be different from the document state on which the operation is executed, the *execution state* of O . We often annotate state S with a subscript, which refers to the site id, and a superscript, which distinguishes different states. For example, an operation O is generated on document state S_i^p and executed on state S_j^q , where S_i^p may or may not be equivalent to S_j^q . Accordingly, to correctly execute the operation, its execution forms on these two document states may not be equivalent. To differentiate, we annotate the execution form of an operation with its definition state. For example, the execution forms of O on S_i^p and S_j^q are $O^{S_i^p}$ and $O^{S_j^q}$, respectively. Given operation O^S , we say that O (more specifically, the position parameter of O) is defined on S and S is the *definition state* of O , regardless of whether S is actually its generation or execution state. The subscript or superscript of a state or operation is omitted when it is not interesting.

2.2. OPERATION RELATIONS AND LOGICAL TIMESTAMPS

Operations are propagated among collaborating sites by passing synchronization messages. Each message takes the form of $\langle V(O), ID(O), T(O), P(O),$

$C(O)$), where $ID(O)$ denotes the id of the site where O is generated, and $V(O)$ denotes the vector timestamp of O , as defined in Ellis and Gibbs (1989). It is worth noting that $C(O)$, $T(O)$, $ID(O)$ and $V(O)$ are inherent attributes of O that are independent of specific states. However, the value of $P(O)$ is state dependent. We annotate O if its definition state is not clearly implied, for example, $P(O^{S_i^p})$.

Every site maintains a log of executed operations, which contains a contextually serialized sequence of operations ordered by their execution times, the leftmost having been executed the earliest. Due to Ellis and Gibbs (1989), we use the following two established notations, causal ordering or precedence relation “ $\rightarrow$ ” and concurrent relation “ $\parallel$ ”: $O_i \rightarrow O_j$ iff O_i is executed or generated before the generation of O_j , and $O_i \parallel O_j$ iff neither $O_i \rightarrow O_j$ nor $O_j \rightarrow O_i$.

Each site i maintains a vector logical clock, its j th element being advanced each time after an operation generated at site j is executed at site i (Ellis and Gibbs, 1989). Every document state is conceptually timestamped with the current value of its vector clock, called state vector (Ellis and Gibbs, 1989). To distinguish, we use function $SV(S)$ to denote the state vector of state S . Every local operation is timestamped by the state vector of the state immediately resulted from its generation (and execution). For example, suppose the initial state vector of a two-site system is $SV(S^0) = (0,0)$. After O_1 is generated (and executed) at site 1, $S_1^0 + O_1 = S_1^1$. We have $V(O_1) = SV(S_1^1) = (1,0)$. After a concurrent operation O_2 is generated at site 2, $S_2^0 + O_2 = S_2^1$. We have $V(O_2) = SV(S_2^1) = (0,1)$. After O_2 is received and executed at site 1, $S_1^1 + O_2^{S_1^1} = S_1^2$. We have $SV(S_1^2) = (1,1)$.

For any two vectors V_1 and V_2 , we say (1) $V_1 = V_2$ if all their corresponding elements are equal, and (2) $V_1 < V_2$ if no element of V_1 is greater than the corresponding element of V_2 and at least one element of V_1 is less than the corresponding element of V_2 . Obviously $O_1 \rightarrow O_2$ if $V(O_1) < V(O_2)$, $O_1 \parallel O_2$ if neither $V(O_1) < V(O_2)$ nor $V(O_2) < V(O_1)$, and $SV(S) < SV(S')$ if $S + SQ = S'$ and sequence SQ has at least one operation. We say $S < S'$ iff $SV(S) < SV(S')$.

2.3. TRANSFORMATION FUNCTIONS

As a convention (Ellis and Gibbs, 1989; Sun et al., 1998), OT algorithms are usually divided into two layers: a control or integration procedure that is application independent, and transformation functions that depend on data structures and operation types. The integration procedure is usually implemented such that local operations are always executed without being delayed but the execution of a remote operation may be delayed until it is causally ready, i.e., all operations that causally precede it have been executed at current site.

The execution form of an operation is usually obtained by transforming it against concurrent operations that have been executed earlier at current site. As a result, the P parameter of the operation is shifted to incorporate the effects of those concurrent operations. Sun et al. (1998) distinguishes the different purposes of transformation functions and defines the concepts of inclusion transformation (IT) and exclusion transformation (ET). $IT(O_1^S, O_2^S)$ transforms an operation O_1^S against another operation O_2^S in such a way that the effect of O_2^S is effectively included. $ET(O_2^{S^1}, O_1^{S^0})$ transforms $O_2^{S^1}$ against $O_1^{S^0}$ in such a way that the effect of $O_1^{S^0}$ is effectively excluded from $O_2^{S^1}$, where $S^0 + [O_1^{S^0}] = S^1$.

Algorithm 1. $IT_II(O_1^S, O_2^S) : O_1^{S'}$, where $S' = S + O_2^S$.
 if $P(O_2^S) < P(O_1^S)$ or $(P(O_1^S) = P(O_2^S) \text{ and } ID(O_1) > ID(O_2))$
 return insert($C(O_1), P(O_1^S) + 1$);
 else
 return O_1 ;

Algorithm 1 gives an example IT rule for two insert operations, where $O_1^S = \text{insert}(P(O_1^S), C(O_1))$ and $O_2^S = \text{insert}(P(O_2^S), C(O_2))$. The reader is referred to Ellis and Gibbs (1989), Sun et al. (1998) for other combinations of IT and (Sun et al., 1998) for the definitions of ET functions. To explain function $IT_II(O_1^S, O_2^S)$, the position parameter of O_1^S is shifted only if a character has been inserted to its left by an operation O_2^S ; or when they were originally intended to insert at the same position, a priority scheme is used to break the tie so that the one with a greater site id is shifted.

2.4. TRANSFORMATION PROPERTIES

Given three operations $O_1^{S^0}, O_2^{S^0}$ and $O_3^{S^0}$, $S^0 + [O_1^{S^0}] = S^1$, $S^0 + [O_2^{S^0}] = S^{1'}$ and $S^0 + [O_3^{S^0}] = S^{1''}$. Let $O_1^{S^{1'}} = IT(O_1^{S^0}, O_2^{S^0})$ and $O_2^{S^1} = IT(O_2^{S^0}, O_1^{S^0})$. Due to Ressel et al. (1996), IT functions are required to hold the following two properties, TP1 and TP2, in order to guarantee convergence along arbitrary transformation paths.

Transformation Property 1 (TP1):

$$S^0 + [O_1^{S^0}, O_2^{S^1}] = S^0 + [O_2^{S^0}, O_1^{S^{1'}}]$$

Transformation Property 2 (TP2):

$$IT(IT(O_3^{S^0}, O_1^{S^0}), O_2^{S^1}) = IT(IT(O_3^{S^0}, O_2^{S^0}), O_1^{S^{1'}})$$

TP1 ensures that if $O_1 \parallel O_2$, the effect of executing O_1 before O_2 is the same as executing O_2 before O_1 . TP2 ensures that transforming any operation O_3 along different paths will yield the same result. These two properties ensure that arbitrary communication orders can lead to a consistent final document state. In other words, it is not necessary to enforce a global total order between operations because inconsistencies can always be repaired with OT. In fact, TP1 and TP2 are the precondition assumed by most existing OT

algorithms, including dOPT (Ellis and Gibbs, 1989), adOPTed (Ressel et al., 1996), GOTO (Sun and Ellis, 1998), and SOCT2 (Suleiman et al., 1998).

3. Consistency problems and analysis

Consider the scenario in Figure 1. Three sites 1, 2, 3 start from the same initial document state, $S_1^0 = S_2^0 = S_3^0 = \text{"abc"}$. They generate operations $O_1 = \text{insert("1",2)}$, $O_2 = \text{insert("2",1)}$, and $O_3 = \text{delete("b",1)}$ concurrently, which change their document states to $S_1^1 = \text{"ab1c"}$, $S_2^1 = \text{"a2bc"}$, and $S_3^1 = \text{"ac"}$, respectively.

At site 2, when O_3 is received, it is inclusively transformed against O_2 , resulting in $O_3^{S_2^1} = \text{delete("b",2)}$. After $O_3^{S_2^1}$ is executed, the state becomes $S_2^2 = \text{"a2c"}$. When O_1 arrives, it is inclusively transformed against $[O_2^{S_2^1}, O_3^{S_2^1}]$, resulting in $O_1^{S_2^2} = \text{insert("1",2)}$, whose execution leads to state $S_2^3 = \text{"a21c"}$.

At site 3, O_2 arrives first and is inclusively transformed against O_3 , resulting in $O_2^{S_3^1} = \text{insert("2",1)}$. After $O_2^{S_3^1}$ is executed, the state becomes $S_3^2 = \text{"a2c"}$. When O_1 arrives, it is inclusively transformed first against O_3 , resulting in $O_1^{S_3^1} = \text{insert("1",1)}$. Then $O_1^{S_3^1}$ is inclusively transformed against $O_2^{S_3^1}$. Since the site id of O_1 is smaller than that of O_2 , it is not shifted. Hence $O_1^{S_3^2} = O_1^{S_3^1} = \text{insert("1",1)}$. After the execution of $O_1^{S_3^2}$, the state of site 3 becomes $S_3^3 = \text{"a12c"}$, which is not consistent with the state $S_2^3 = \text{"a21c"}$ of site 2.

The above scenario is due to Suleiman et al. (1998) and Sun et al. (1998). As analyzed in Sun et al. (1998), the problem cannot be fixed by simply reversing the site ids of O_1 and O_2 , because this quick fix works here but will

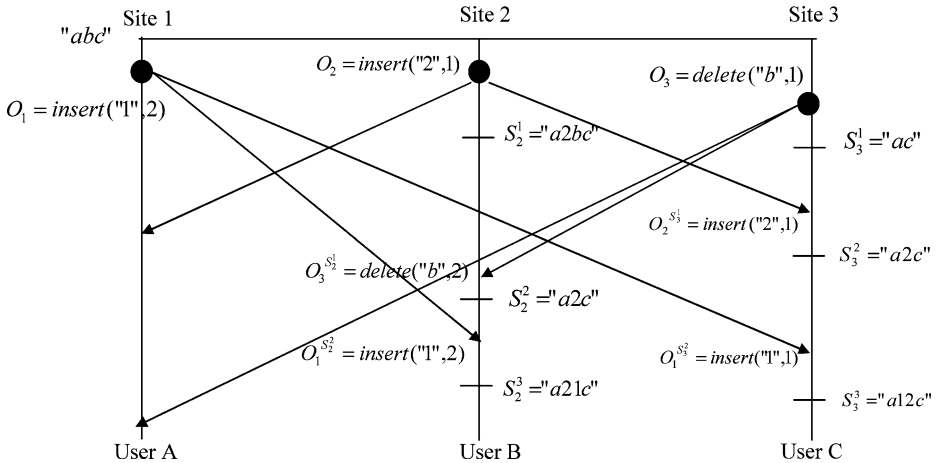


Figure 1. A scenario of two consistency problems: all three sites start with the same initial state "abc". After all generated operations are executed, the states of sites 2 and 3 are divergent, and the state of site 3 fails to maintain the correct operation effects relation.

fail in other similar scenarios. This scenario reveals two consistency problems of different nature: First is divergence. After O_1 , O_2 and O_3 are executed at sites 2 and 3 in different orders (on the same initial state), the final states are not identical. The cause of this problem is that, after $O_1^{S^0}$ is inclusively transformed along different paths $[O_2^{S^0}, O_3^{S^1}]$ and $[O_3^{S^0}, O_2^{S^1}]$ at sites 2 and 3, respectively, different results are obtained, i.e., $O_1^{S^2} = \text{insert}("1", 2)$ and $O_1^{S^3} = \text{insert}("1", 1)$. That means the IT functions cannot satisfy the TP2 condition defined above. The second problem is operation effects relation violation. If the landmark character 'b' had not been deleted, intuitively, the operation effect of O_2 ('2') should have been to the left of effect of O_1 ('1'). However, the final state at site 3 obviously violates this correct effects relation.

When processing $\text{IT}(O_1^{S^1}, O_2^{S^1})$ at site 3, if we had somehow known that O_2 was intended to insert a character to the left of O_1 's intended effect position relative to S^0 , we would have shifted the position parameter of $O_1^{S^1}$ to get the right result $O_3^{S^2} = \text{insert}("1", 2)$. Then all sites would have converged to the same effects relation preserving state, and then the TP2 puzzle would have not been a problem. The question is how to capture the intended effects relation.

Our intuition comes from the fact that the position of every character in a string is unique. That is, assuming that every appearance of a character has a unique identity, no two characters in a string take the same position. Therefore the position relation between all characters in any string is a total order. We denote such a total order as $\prec$. Apparently relation $\prec$ is transitive. For example, in the initial state $S^0 = \text{"abc"}$ of Figure 1, we have $'a' \prec 'b' \prec 'c'$. Due to $'b' \prec '1' \prec 'c'$ in state $S_1^1 = \text{"ab1c"}$ and $'a' \prec '2' \prec 'b'$ in state $S_2^1 = \text{"a2bc"}$, we have $'a' \prec '2' \prec 'b' \prec '1' \prec 'c'$.

Characters in any document state are eventually a result of editing operations. Every characterwise operation O has an effect character $C(O)$. Hence we extend the total order $\prec$ of characters to operations. Given any two operations O_x and O_y , we say $O_x \prec O_y$, iff $C(O_x) \prec C(O_y)$. For example, in Figure 1, we have $O_2 \prec O_3 \prec O_1$.

At site 2, the execution of O_2 is straightforward since its execution state is equivalent to its generation state. However, after executing O_2 , we cannot execute O_1 directly as it is on the current state. This is because the position value of O_1 is defined on a document state before O_2 is executed. The solution is to adjust the execution position of O_2 by the effects relation between O_1 and O_2 . From the above analysis, we know that $O_2 \prec O_1$, which means the execution of O_2 has changed the definition state of O_1 . Because O_2 has inserted one character to the left of O_1 , we shift the position of O_1 to incorporate the effect of O_2 such that the execution form of O_1 , $\text{insert}("1", 3)$, can be correctly executed in the current state.

The basic idea of OT is to shift the position parameter of an operation to incorporate the effects of previously executed concurrent operations so that the transformed form can be correctly executed on the current state. The precondition that the position parameter can be correctly adjusted is that the IT functions can correctly determine the position relation between the effect characters of the two involved operations. Unfortunately, in the above case and the alike, existing IT functions cannot correctly determine the effects relation.

Specifically, the original IT functions of Ellis and Gibbs (1989) try to determine the effects relation by two parameters, $P(O)$ and $ID(O)$. However, the method cannot work correctly in the above scenario. At site 3 in Figure 1, when $O_1^{S_1^3}$ is inclusively transformed against $O_2^{S_1^3}$, we have $P(O_1^{S_1^3}) = P(O_2^{S_1^3})$ and $ID(O_1) < ID(O_2)$. As a result, character '1' is inserted before '2', which violates the actual effects relation ' $2 \prec 1$ ' and causes the states at different sites to diverge.

Most OT algorithms such as GOT (Sun et al., 1998), GOTO (Sun and Ellis, 1998), and adOPTed (Ressel et al., 1996) are based on the original IT functions defined by Ellis and Gibbs (1989). Hence they are not able to solve the problem. Imine et al. (2003) have also found the defect of the IT functions of Ellis and Gibbs (1989). Unfortunately, their new IT functions also cannot correctly find out the operation effect order in the TP2 puzzle scenarios. Our approach to be presented in this article will follow the same direction of Suleiman et al. (1998) and Imine et al. (2003) to extend IT functions so that they can correctly identify the effects relation between concurrent operations.

The SDT algorithm is the first (and the only) OT algorithm that is proved to achieve convergence along arbitrary communication and transformation paths. This is a desirable property because it greatly simplifies the design of group editors. In other words, group editors that implement this algorithm are allowed to be peer-to-peer and are not required to execute a central component for ensuring convergence. As will be discussed in Section 11, this property has not been achieved in previous OT algorithms including GOTO (Sun and Ellis, 1998; Sun, 2002b) and SOCT3/4 (Vidot et al., 2000).

4. The CSM consistency model

Operation effects relation reflects the intentions of the users and their operations to some degree. In Figure 1, based on the landmark character 'b', it is obvious that the intended effect (or intention) of O_1 is to insert '1' after 'b' and the intention of O_2 is to insert '2' before 'b'. At the system level, due to the lack of information about user intentions, we do not know whether or not the result "a21c" really preserves the intentions of O_1 and O_2 or the users who issued the operations. However, it is clear that result "a12c" are consistent

with the intentions of neither. Unfortunately, according to Sun et al. (1998), both results are intention-preserving. Therefore it seems necessary to revisit the convergence, causality and intention-preservation (CCI) model defined in Sun et al. (1998), or at least its intention preservation part.

In this section we formally define the concept of operation effects relation. Based on this concept, we then define our CSM consistency model and distinguish it from the accepted CCI model.

4.1. RELATION OF OBJECTS AND OPERATION EFFECTS

As a convention, we model any document state as a linear string (or a sequence of objects in general). The position relation of characters in any string is a total order, i.e., no two different characters will take the same position in any document state. We introduce the notation $\prec_S$ to represent the position relation of characters relative to a given state S . Mathematically relation $\prec$ is a set. For any two characters c_1 and c_2 that both appear in state S , we say $c_1 \prec_S c_2$ iff the position of c_1 is to the left of the position of c_2 , or $P(c_1) < P(c_2)$, in state S . Apparently this relation is transitive. That is, if $c_1 \prec_S c_2$ and $c_2 \prec_S c_3$, we have $c_1 \prec_S c_3$.

In the following, for the reader to understand this relation in states that are empty or have only one character, we can imagine that there are two special invisible characters: c_b marks the beginning of the document, and c_e marks the end of the document. Apparently we have $c_b \prec_S c_e$ for any state S and $c_b \prec_S c_x \prec_S c_e$ for any “normal” character c_x .

With the execution of editing operations, the document is transformed from one state to another. Some characters are inserted and some deleted. As in all other group editors, we assume (1) that every appearance of a character has a unique identity, and (2) that the execution of any operation in any state does not change the total order of existing characters in that state.

Observe, for example, in state $S^0 = \text{“abc”}$, we have $\text{‘a’} \prec_{S^0} \text{‘b’} \prec_{S^0} \text{‘c’}$. The execution of operation $O_1^{S^0} = \text{insert(“x”,1)}$ in state S^0 results in state $S^1 = \text{“axbc”}$ and introduces relation $\text{‘a’} \prec_{S^1} \text{‘x’} \prec_{S^1} \text{‘b’} \prec_{S^1} \text{‘c’}$. However, except the part involving the new character ‘x’, the other relations are inherited from state S^0 . Now suppose an operation $O_2^{S^1} = \text{delete(“b”,2)}$ is executed in state S^1 , which results in state $S^2 = \text{“axc”}$. We have relation $\text{‘a’} \prec_{S^2} \text{‘x’} \prec_{S^2} \text{‘c’}$, which is consistent with relation $\prec$ between characters ‘a’, ‘x’, and ‘c’ relative to state S^1 .

Based on the above observation, we define the character position relation $\prec$ of all characters that ever appear in a group editors as a union of all relation $\prec_{S_i^p}$ for any i and p . Using operator $\cup$ for set union, we have $\prec = \bigcup_i \bigcup_p \prec_{S_i^p} = \bigcup_p \bigcup_i \prec_{S_i^p}$.

Definition 1. For any characters c_1 and c_2 , we say $c_1 \prec c_2$, iff (1) there exists a state S such that $c_1 \overset{S}{\prec} c_2$, or (2) there exists another character c_3 such that $c_1 \prec c_3$ and $c_3 \prec c_2$.

Under the two assumptions stated above, it often suffices to only consider some subset of $\prec$ because the character position relations appear in any subset must be consistent with $\prec$. For example, in Figure 1, we only need to consider the following subset: $\{ 'a' \overset{S^0}{\prec} 'b' \overset{S^0}{\prec} 'c', 'a' \overset{S^1_1}{\prec} 'b' \overset{S^1_1}{\prec} '1' \overset{S^1_1}{\prec} 'c', 'a' \overset{S^1_1}{\prec} '2' \overset{S^1_2}{\prec} 'b' \overset{S^1_2}{\prec} 'c', 'a' \overset{S^1_3}{\prec} 'c' \}$. Then from it we can infer that $'a' \prec '2' \prec 'b' \prec '1' \prec 'c'$. Based on this subset, even without direct knowledge between the two concurrently inserted characters '2' and '1', we can still derive relation $'2' \prec '1'$.

However, it is important to note that the position relation is not always as clear as the above example. Consider, for example, $S^0 = "ab"$ and two characters 'x' and 'y' are inserted between 'a' and 'b' by two concurrent operations O_1 and O_2 ($O_1 \parallel O_2$). We have $'a' \overset{S^0}{\prec} 'b', 'a' \overset{S^1_1}{\prec} 'x' \overset{S^1_1}{\prec} 'b',$ and $'a' \overset{S^1_2}{\prec} 'y' \overset{S^1_2}{\prec} 'b'$. As another example, suppose $S^0 = "axb"$. Character 'x' is deleted by operation O_1 , which results in $S^1 = "ab"$. Then another character 'y' is inserted between 'a' and 'b' by a causally following operation O_2 ($O_1 \rightarrow O_2$). We have $'a' \overset{S^0}{\prec} 'x' \overset{S^0}{\prec} 'b', 'a' \overset{S^1}{\prec} 'b',$ and $'a' \overset{S^2}{\prec} 'y' \overset{S^2}{\prec} 'b'$. In these two cases, what should be the position relation between 'x' and 'y'?

Due to the lack of knowledge about the actual user intentions and application context, we have to assume that either $'x' \prec 'y'$ or $'y' \prec 'x'$ is acceptable at the system level. Eventually some priority scheme has to be adopted to break the tie. For example, in the first case, we may consistently enforce that the character inserted by the operation with a smaller site id totally precedes the other character. In the second case, we may enforce that the character inserted by O_2 always totally precedes the character deleted by O_1 . As a result, relation $\prec$ remains a total order between all characters that ever appear in any state.

In a group editing session, all sites start from the same initial state. As editing operations are performed, characters are inserted and deleted to form a sequence of states at each site, which adds more relations to set $\prec$. Insertion and deletion of characters change the positions of characters but not their relations. Characters that are deleted from a state do not take positions in subsequent states but their relations with other characters carry over. However, the execution of new operations should not alter the existing character position relation due to our assumption. This has two implications: First, we can use relation $\prec$ to check if an execution is consistent, i.e., whether the new relations generated by this execution are consistent with what are there. Second, from earlier relations we can consistently derive new relations involving newly inserted characters.

For example, in Figure 1, on one hand, the execution at sites 2 and 3 introduces two contradicting relations, ' $2 \prec_{S_2^3} 1$ ' and ' $1 \prec_{S_3^3} 2$ '. Hence we can conclude that algorithms that produce this execution are not correct. On the other hand, at site 3, when we need to decide the position relation between '1' and '2' in state S_3^2 , if we can correctly derive ' $2 \prec 1$ ' as in the above analysis, then there will be no doubt that we can get the correct result.

Observe that any character appears in a given document state must be the effect of some operation. Hence based on the above total order $\prec$ between characters, we define the effects relation between any two operations as follows.

Definition 2. For any two operations O_1 and O_2 , we say $O_1 \prec O_2$ iff $C(O_1) \prec C(O_2)$. We say the effect characters of O_1 and O_2 are equivalent, denoted as $C(O_1) \equiv C(O_2)$, if (1) they are inverse operations, meaning one deletes the character inserted by the other, or (2) they concurrently delete the same character.

4.2. CONSISTENCY MODEL DEFINITION

A group editor is CSM-consistent iff the following three conditions hold:

- (1) Causality preservation: all operations are executed in their cause-effect order. To verify, for any two operations O_1 and O_2 , if $O_1 \rightarrow O_2$, then O_1 is executed before O_2 at all sites.
- (2) Single-operation effects preservation: the effect of executing any operation in any execution state achieves the same effect as in its generation state. To verify, suppose operation O is generated in state S_i^p and executed in state S_j^u . If O is a deletion, the same character is deleted from both states; or if O is an insertion, the same total order maintains between any two characters that appear in both resulted states S_i^{p+1} and S_j^{u+1} .
- (3) Multi-operation effects relation preservation: the effects relation of any two operations maintains after they are both executed in any states. To verify, given any two operations O_i and O_j that are generated in states S^i and S^j , respectively, without loss of generality, assume $C(O_i) \prec C(O_j)$. The same effects relation holds after they are both executed in any state S^k .

The preservation of single-operation effect does not necessarily guarantee that the effect relation of two concurrent operations can be correctly maintained. For example, in Figure 1, the effect of either O_1 or O_2 is correctly preserved at site 3. Concretely speaking, the effect of O_1 is to insert '1' after 'b' and before 'c', and the effect of O_2 is to insert '2' after 'a' and before 'b'. At site 3 the execution of O_2 results in state $S_3^2 = \text{"a2c"}$. Although 'b' does not survive in S_3^2 , due to our above definition, it is obvious that the effect of O_2 is preserved in

S_3^2 . Similarly, after the execution of O_1 , the effect of O_1 is also preserved. It is worth noting that the execution of O_1 and O_2 introduces a new position relation between '1' and '2' which does not exist in S_3^0 . The correct position relation between '1' and '2' should be "21" (or '2' $\prec$ '1'). Unfortunately, this position relation is not maintained in result "a12c", although the effects of either O_1 or O_2 is correctly preserved. This justifies the need for preserving multi-operation effects relation in addition to single-operation effects.

4.3. SYNTACTIC AND SEMANTIC CONSISTENCY

The CCI model (Sun et al., 1998) is well-accepted in group editors, e.g. (Bellini et al., 2002; Sun and Chen, 2002). The causality preservation condition is inherited because it is generally required in distributed systems. We drop the convergence requirement because it is automatically satisfied once single-operation effects and multi-operation effect relations are preserved, as is stated in the following theorem.

Theorem 1. *Single-operation effect preservation and multi-operation effect relation preservation imply convergence in quiescent states.*

Proof. In a quiescent state, after all sites have executed all operations, all involved sites will have the same set of characters in the same order. This is because, due to single-operation effects preservation, each operation alone inserts or deletes its effect character at the right position at all sites, which at the same time does not change the position order of existing characters. In addition, due to multi-operation effect relation preservation, all new characters introduced by concurrent insert operations maintain the same order at all sites. Hence these two conditions together ensure that all sites have the same set of characters that are of the same order, i.e., the states of all sites converge. $\square$

Intention preservation is a tricky issue in group editors as well as general groupware applications. Due to Dourish (1996), there are approximately two kinds of consistency problems: *syntactic consistency* only requires that all sites have an identical view of the shared document, while *semantic consistency* additionally demands that the same view make sense in the application context. The former is usually termed as convergence, and the latter is also termed as intention preservation (Sun et al., 1998; Sun and Ellis, 1998).

Intention preservation in Sun et al. (1998) is defined as (1) that the effect of executing any operation O at remote sites achieves the same effect as executing O at the local site at the time of its generation, and (2) that the execution effects of concurrent operations do not interfere with each other. Verification methods for (1) are given in Sun et al. (1998) and Sun (2002b)

but have been absent for (2). The first condition is inherited in our notion of single-operation effects preservation. The second condition seems more general than our notion of multi-operation effects relation preservation but less formalized. It does not clearly specify exactly under which condition concurrent execution does or does not *interfere* each other.

In fact, concurrent operations do interfere each other unless “properly” coordinated. A classic example is that two users attempt to fix the grammatical error in sentence “There should be student.” at the same time, one trying to insert “a” before and another to insert “s” after “student”. Current group editors will produce a syntactically consistent but semantically inconsistent result “There should be a students”. However, a rigorous definition of semantic consistency has been absent in the literature, although it is understood that grammatical correctness should be among the requirements (Sun et al. 1998). The characterization and preservation of user intentions or semantic consistency still remains an open issue (Sun and Ellis, 1998). Apparently semantic consistency cannot be easily formalized and verified by pure system approaches.

Hence the CSM model reflects a more realistic approach to addressing consistency problems in group editors. It ensures syntactic consistency and the part of semantic consistency (operation effects relation) that is already well-understood, while deliberately leaving general semantic consistency to the interpretation of end users. As an integrated and active part of the collaborative system, users will be able to maintain semantic consistency and preserve their intentions with further helps from, e.g., mechanisms for error recovery (Sun, 2002b), optional locking (Sun, 2002a), and multi-versioning (Sun and Chen, 2002; Sun et al., 2004), real-time communication and awareness (Greenberg and Marwood, 1994), and coordination protocols (Backer et al., 1993; Noel and Robert, 2004).

The fact that group editors allow users to edit the same part of the shared document at the same time does not necessarily imply that users have to work that way. It makes perfect sense for users to work on different parts at the same time and take turns to work on the same part, e.g., by following social protocols. The question is how to “properly” coordinate individual editing activities. On one hand, formalizing protocols into the system is generally difficult and often suffers from the dangers of being inflexible (Grudin, 1994; Shipman and Marshall, 1999). On the other hand, a system that totally relies on the users’ conscious following of implicit protocols or processes may risk laying too much burden on its users, although significant flexibility can be achieved (Edwards, 1996).

Argument about different coordination paradigms and implementation of specific coordination mechanisms are beyond the scope of this article. However, the focus of group editors has been traditionally on supporting familiar small groups (Ellis and Gibbs, 1989), in which people usually

coordinate well by social protocols (Baecker et al., 1993). It makes sense and is often more effective in this context to rely on social protocols, which is able to easily accommodate a variety of work styles and writing processes while avoiding the costs and problems of formalization. It is generally assumed that, with real-time communication and awareness, users can easily discover and resolve conflicts and semantic inconsistencies in a timely manner (Ellis and Gibbs, 1989; Greenberg and Marwood, 1994; Sun et al., 1998).

As argued in Bentley and Dourish (1995a), collaborative systems such as group editors should be built as a collaboration medium, without constraining the possible customized usages by end users. OT seems more promising among alternative technologies (Ellis et al., 1991; Sun et al., 1998) in building such a responsive and unconstraining collaboration medium, the use of which enables end users to make sense of their individual operations in the context of specific group editing tasks. This separation of concerns is significant in groupware engineering because the evaluation of the whole system then becomes more tractable: the correctness of the medium can be evaluated by pure system methods such as proof and verification, while the interface features can be evaluated by usability methods that are often less definitive (Grudin, 1994).

5. Discussion of IT

We use OT for achieving CSM consistency in group editors. As a convention, we use vector timestamps for achieving causality preservation (Ellis and Gibbs, 1989). For responsiveness, local operations are always executed as soon as they are generated. Every causally ready remote operation is (inclusively) transformed against those that are concurrent with it and that have been executed. Intuitively, if the effects relation of all operations are known, the execution of any remote operation will be able to achieve both single-operation effects preservation and multi-operation effects relation preservation. Then convergence can then be achieved regardless of transformation paths. Hence the problem of achieving CSM consistency is reduced to how to compute relation $\prec$.

The above consistency model serves as a theoretical basis for interpreting and verifying OT algorithms. On one hand, numerous possible approaches and algorithms could be motivated from this model. Our approach, state difference (SD) based transformation or SDT, is only one such possibility. On the other hand, however, the effects relation of operations as inferred above is of little practical value due to performance reasons. It would be extremely inefficient to track and infer the position relation between all characters and then reason about the operation effects relation in a distributed system.

In this section, we reinterpret IT with our new notion of operation effects relation $\prec$. We conceptually discuss how to derive relation $\prec$ between

concurrent operations. Our method is based on a common previous state of the two concurrent operations, called the latest synchronization point (LSP). Then Section 6 will reinterpret ET and discuss the possibility and problems of using ET for deriving relation $\prec$. Section 7 will elaborate our SDT algorithm for computing relation $\prec$. We then give our control algorithm in Section 8 and prove that our approach ensures CSM consistency in Section 10.

5.1. IT REINTERPRETED

Algorithm 2. $IT(O_1^S, O_2^S)$: $O_1^{S'}$, where $S' = S + O_2^S$ and $O_1 \parallel O_2$.

```

 $O_1^{S'} := O_1^S$ ;
if  $O_2 \prec O_1$ 
  if  $T(O_2) = \text{INS}$ 
     $P(O_1^{S'}) := P(O_1^S) + 1$ ;
  else  $//T(O_2) = \text{DEL}$ 
     $P(O_1^{S'}) := P(O_1^S) - 1$ ;
return  $O_1^{S'}$ ;
```

Assuming that relation $\prec$ is known, we define the IT function conceptually as in Algorithm 2. When O_1 is inclusively transformed against O_2 , they must be defined in the same state and O_2 must have been executed (Sun et al., 1998). Only when $O_2 \prec O_1$, i.e., the effect position of O_2 should be to the left of that of O_1 , do we need shift the position of O_1 . If O_2 is an insertion, we shift the position of O_1 by one position to the right; Or if O_2 is a deletion, we shift the position of O_1 by one position to the left. Note $P(O)$ returns the position of O in its definition state.

5.2. LATEST SYNCHRONIZATION POINT

The problem is how to determine the effects relation $\prec$. Clearly due to the counterexample in Figure 1 comparing the position parameters and site ids of operations alone is insufficient for this purpose. In Figure 1, when processing $IT(O_1^{S_3}, O_2^{S_3})$ at site 3, because $P(O_1^{S_3}) = P(O_2^{S_3})$, the original IT functions use site ids to break the tie and get the wrong result. Unfortunately, almost all previous OT algorithms (except Suleiman et al., 1998; Imine et al., 2003) are based on the IT functions of Ellis and Gibbs (1989) which use position parameters to determine the effects relation and resort to site ids if there is a tie.

The comparison between $P(O_1^{S_3})$ and $P(O_2^{S_3})$ is based on a state in which a common third operation O_3 has been executed. What if we go back to a prior state in which O_3 is not executed? The natural choice here is the initial state S^0 . Without the effect of O_3 , we know that the effect position of $O_1^{S^0}$ is 2 and that of $O_2^{S^0}$ is 1. That is, $O_2 \prec O_1$.

To extend this intuition into a general solution, however, we need to answer two questions. First, which previous state S' shall we examine when processing $IT(O_1^S, O_2^S)$ between any two concurrent operations O_1 and O_2 ? Second, how to calculate their effect positions relative to S' ?

To answer the first question, apparently S' must be a common previous state of both O_1^S and O_2^S . Otherwise, even after we calculate their effect positions relative to two different previous states, it makes no sense to compare them. Suppose that S^0 is the initial state at all sites, and O_1 and O_2 are generated in state S_i^p and state S_j^u , respectively. Then due to causality preservation, S_i^p and S_j^u must also be prior to current state S .

Let SQ be a sequence of n operations $O_1, O_2, \dots, O_n$, the execution of which in state S^1 results in state S^n , or $S^1 + SQ = S^n$. We denote the relation that S^1 is prior to S^n as $S^1 < S^n$ if $n > 0$. We say $S^1 \leq S^n$ if $n \geq 0$. Continuing with the above analysis, obviously we have $S^0 \leq S' \leq S_i^p \leq S$ and $S^0 \leq S' \leq S_j^u \leq S$, but S_i^p may not be equal to S_j^u . The relation between states is only a partial order.

Hence we may not really need to go back to state S^0 . Instead, a possibly later state S' that is “closest” to both S_i^p and S_j^u would suffice and save some computation. We introduce the concept of *latest synchronization point* to represent such a state S' . Let V_1 and V_2 be the vector timestamps of O_1 and O_2 , respectively. Let SV_{min} be a state vector, each element of which is equal to the minimal value of the corresponding elements of V_1 and V_2 . Then S^{lsp} , the state corresponding to SV_{min} , is what we want.

5.3. DETERMINING THE OPERATION EFFECTS RELATION

We introduce the notation of $\beta_{S'}(O)$ to represent the effect position of operation O^S relative to any state S' that is prior to its definition state S . Let the effect positions of O_1 and O_2 relative to state S^{lsp} be $\beta_{lsp}(O_1)$ and $\beta_{lsp}(O_2)$, respectively. The algorithm for computing β_{lsp} will be given in Section 7. We define function $O_1 \prec O_2$ in Algorithm 3. We postpone the use of the priority scheme until both their β and P values are tied. Note that $P(O_1)$ and $P(O_2)$ are relative to the current state S , that is, the state in which $IT(O_1^S, O_2^S)$ or $IT(O_2^S, O_1^S)$ is processed.

Algorithm 3. $O_1^S \prec O_2^S$: boolean, where $O_1 \parallel O_2$.

```

if  $\beta_{lsp}(O_1^S) < \beta_{lsp}(O_2^S)$  return true;
else if  $\beta_{lsp}(O_1^S) = \beta_{lsp}(O_2^S)$ 
  if  $P(O_1^S) < P(O_2^S)$  return true;
  else if  $P(O_1^S) = P(O_2^S)$ 
    if  $T(O_1) = \text{INS}$  and  $T(O_2) = \text{DEL}$  return true;
    else if  $T(O_1) = T(O_2) = \text{INS}$  and  $\text{ID}(O_1) < \text{ID}(O_2)$ 
      return true;
return false; //meaning  $O_2^S \prec O_1^S$ 
```

Algorithm 4. $IT_II(O_1^S, O_2^S)$: O_1^S , where $O_1 \parallel O_2$ and $S' = S + O_2^S$.

1. if $\beta_{lsp}(O_1) < \beta_{lsp}(O_2)$ return O_1 ;
2. else if $\beta_{lsp}(O_1) > \beta_{lsp}(O_2)$ return insert($C(O_1), P(O_1) + 1$);
3. else // $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$
4. if $P(O_1) < P(O_2)$ return O_1 ;
5. else if $P(O_1) > P(O_2)$ return insert($C(O_1), P(O_1) + 1$);
6. else // $P(O_1) = P(O_2)$
7. if $ID(O_1) < ID(O_2)$ return O_1 ;
8. else // $ID(O_1) > ID(O_2)$
9. return insert($C(O_1), P(O_1) + 1$);

The IT function for transforming between two insert operations is redefined by our β notation in Algorithm 4. IT functions for other operation combinations can be defined similarly. It is easy to see that the original IT functions of Ellis and Gibbs (1989) (e.g., Algorithm 1) is a special case of our IT function (Algorithm 2). As in previous work (e.g., Ellis and Gibbs, 1989; Sun et al., 1998), when transforming two concurrent operations that aim to delete the same character, we always return an identity operation.

5.4. OPERATION EFFECTS IN PRIOR STATES

In a group editor, the generation state and execution state of an operation are often not the same. The effect of an operation O on its generation state S^i is obvious. However, the effect of operation O^{S^j} on another state S^i is usually not as obvious if $S^i < S^j$. Consider the following example. Assume two operations, $O_1^{S^0} = \text{insert}(\text{"x"}, 2)$ and $O_2^{S^1} = \text{insert}(\text{"y"}, 4)$, are executed on state $S^0 = \text{"abcd"}$ in order, where $S^0 + [O_1^{S^0}] = S^1$. After $O_1^{S^0}$ is executed on S^0 , the state becomes $S^1 = \text{"abxcd"}$. After $O_2^{S^1}$ is executed on S^1 , the state becomes $S^2 = \text{"abxcyd"}$. The effect of O_2 on S^1 is to insert 'y' at the position 4. How about the effect of O_2 on S^0 ? We cannot know it directly because of the execution of O_1 in the middle. To characterize the effect of an operation on a prior document state more accurately, we introduce two notations, β and δ , as follows. Conceptually, $\beta(O)$ is $P(O)$ relative to a prior state S , and $\delta(O)$ is an auxiliary parameter that distinguishes operations with the same β values. The actual algorithm will be given later in Section 7.

Definition 3. (β and δ of insert) Given a document state S^i , an operation $O^{S^j} = \text{insert}(C, P)$, where $S^i \leq S^j$. If $S^j[P-1]$, the $(P-1)$ th character of S^j , was originally the Q th character in S^i , then $\beta_{S^i}(O^{S^j}) = Q + 1$ and $\delta_{S^i}(O^{S^j}) = 0$. Otherwise, search S^j from $S^j[P-1]$ backward until the first character ($S^j[U]$) that was originally the R th character in S^i . Then $\beta_{S^i}(O^{S^j}) = R + 1$ and $\delta_{S^i}(O^{S^j}) = (P-1) - U$. If there is not such a character, then $\beta_{S^i}(O^{S^j}) = 0$ and $\delta_{S^i}(O^{S^j}) = P$.

Definition 4. (β and δ of delete) Given a document state S^i , an operation $O^{S^j} = \text{delete}(C, P)$, where $S^i \leq S^j$. If $S^j[P]$, the P th character of S^j , was originally the Q th character in S^i , then $\beta_{S^i}(O^{S^j}) = Q$ and $\delta_{S^i}(O^{S^j}) = 0$. Otherwise, search S^j from $S^j[P-1]$ backward until the first character ($S^j[U]$) that was originally R th character in S^i . Then $\beta_{S^i}(O^{S^j}) = R+1$ and $\delta_{S^i}(O^{S^j}) = P-U$. If such a character is not found, then $\beta_{S^i}(O^{S^j}) = 0$ and $\delta_{S^i}(O^{S^j}) = P+1$.

We characterize the effect of operation O^{S^j} relative to state S^i by $\beta_{S^i}(O^{S^j})$ and $\delta_{S^i}(O^{S^j})$. Obviously the effect of an operation O relative to its own definition state S is $\beta_S(O) = P(O)$ and $\delta_S(O) = 0$. In the above example, we have $\beta_{S^0}(O_1) = 2$ and $\delta_{S^0}(O_1) = 0$, $\beta_{S^0}(O_2) = 3$ and $\delta_{S^0}(O_2) = 0$. If $O_3 = \text{delete}("x", 2)$ is executed on S^2 , resulting in $S^3 = "abcd"$, then O_3 does not really have any effect on S^0 since it counteracts a prior operation. By definition we have $\beta_{S^0}(O_3) = 2$ and $\delta_{S^0}(O_3) = 1$. Further, if $O_4 = \text{insert}("z", 4)$ is executed on S^3 , resulting in $S^4 = "abczyd"$, then we have $\beta_{S^0}(O_4) = 3$ and $\delta_{S^0}(O_4) = 1$.

To explain how Algorithm 3 is used for determining the effects relation of two concurrent operations, we give two examples as follows. First consider how O_1 is integrated at site 3 in Figure 1. To compute $O_1^{S_3^1}$, we need to process $\text{IT}(O_1^{S_3^1}, O_2^{S_3^1})$. What is the effects relation between O_1 and O_2 ? Based on the above definition, since the LSP of O_1 and O_2 is S^0 , we have $\beta_{S^0}(O_1) = 2$ and $\beta_{S^0}(O_2) = 1$. By comparing the β values, it is easy to see that $O_2 \prec O_1$. Based on this relation, we can obtain the correct form of $O_1^{S_3^2}$, $\text{insert}("1", 2)$. Note that it is different from the result $\text{insert}("1", 1)$ obtained by using the IT functions defined in Ellis and Gibbs (1989).

As another example see Figure 2. The initial state at both sites are the same: $S_1^0 = S_2^0 = "abc"$. Let $O_1 = \text{insert}("x", 1)$, $O_2 = \text{insert}("y", 1)$, and $O_3 = \text{insert}("z", 2)$. At site 2, we have $S_2^0 + [O_2, O_3] = S_2^1 + [O_3] = S_2^2 = "ayzbc"$.

Now examine how O_1 is integrated at site 2. When O_1 arrives, since both O_2 and O_3 are concurrent with O_1 , O_1 is first inclusively transformed against O_2 . To determine the effects relation of O_1 and O_2 , we first compare their β values based on their LSP, i.e., S^0 . It is obvious that $\beta_{S^0}(O_1) = \beta_{S^0}(O_2)$.

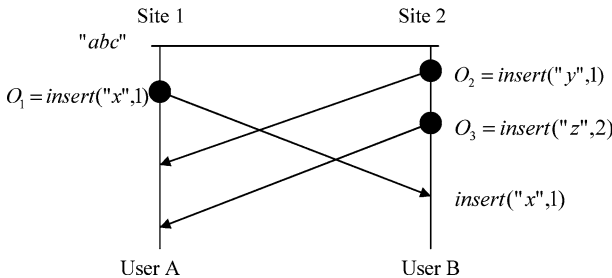


Figure 2. Two users are editing a document with the same initial state "abc".

Then we compare their P values based on S^0 , which are also the same. Thus we must resort to their site ids to break this tie. Since the site id of O_1 is less than that of O_2 , we have $O_1 \prec O_2$. As a result, $O_1^{S_1^1} = \text{insert}(\text{"x"}, 1)$.

Then how about the result of $O_1^{S_2^1} = \text{IT}(O_1^{S_1^1}, O_3^{S_1^1})$? We also first compare their β values based on their LSP to determine the effects relation. The LSP of O_1 and O_3 is S^0 . Hence $\beta_{S^0}(O_1) = \beta_{S^0}(O_3)$. We continue to compare their P values based on S_2^1 . Since $P(O_1^{S_1^1}) < P(O_3^{S_1^1})$, we have $O_1 \prec O_3$. As a result, $O_1^{S_2^1} = \text{insert}(\text{"x"}, 1)$. After $O_1^{S_2^1}$ is executed, the document state becomes "axyzbc".

At site 1, the integration of operation O_2 is straightforward because O_1 and O_2 are defined on the same state S^0 . However, when O_3 arrives, we cannot inclusively transform O_3 against O_1 or compute their effects relation directly by Algorithms 2 and 3, because they are not defined on the same state. This is a typical dOPT puzzle scenario (Sun and Ellis, 1998). The solution will become clear in Section 8.

6. Discussion of ET

As revealed in last section, the intuition of our solution is, when transforming two concurrent operations, we try to compute their operation effects (β) relative to a common previous state, i.e., their LSP. Then by their β values and the tie-breaking rules, we can unambiguously determine their effects relation. We only give a conceptual definition of β in last section. As will be elaborated in Section 7, the actual computation of β is based on ET. To compute $\beta_S(O)$ relative to a prior state S , we must exclusively transform O against all operations that are executed on S before O so that their effects are effectively excluded from O .

Although ET is a reverse of IT by definition, it cannot always be correctly performed. We analyze the problems of ET as defined in Sun et al. (1998). It is important to note that these problems have been avoided in later work of Sun and Ellis (1998), Sun (2002b) because ET will only be performed between concurrent operations. However the use of ET is not limited to concurrent operations in our work. Hence it is still important to understand and address these problems.

6.1. ET REINTERPRETED

Given a state S' prior to the current state S , where $S' + SQ = S$ and $SQ = [O_1, O_2, \dots, O_n]$, the established way to compute $\beta_{S'}(O)$, the effect position of operation O relative to S' , is to use ET to exclude the effects of sequence SQ from S . That is, to perform $\text{ET}(O^S, [O_n, \dots, O_2, O_1])$. According to Sun et al. (1998), exclusively transforming O against a contextually serialized sequence SQ must be processed in reverse order. Similarly to the IT

function in Algorithm 2, we define the conceptual ET function in Algorithm 5 based on effects relation $\prec$.

Algorithm 5. $ET(O^{S^i}, O_i^{S^{i-1}}) : O^{S^{i-1}}$, where $S^i = S^{i-1} + O_i^{S^{i-1}}$.

```

 $O^{S^{i-1}} := O^{S^i};$ 
if  $O_i \prec O$ 
  if  $T(O_i) = \text{INS}$ 
     $P(O^{S^{i-1}}) := P(O^{S^{i-1}}) - 1;$ 
  else //  $T(O_i) = \text{DEL}$ 
     $P(O^{S^{i-1}}) := P(O^{S^{i-1}}) + 1;$ 
return  $O^{S^{i-1}};$ 

```

6.2. DETERMINING RELATION $\prec$ IN ET

Again the problem becomes how to determine the effects relation between the two operations in question. Differently from IT (Algorithm 2), however, we cannot always resort to their β values, because LSP is only defined for concurrent operations. The ET functions defined in Sun et al. (1998) decides the effects relation of operations by comparing their position parameters. This approach works correctly only when the relation of their P values can correctly match the actual effect order of the operations.

In general some operations in SQ may happen before O and some may be concurrent with it. For those operations in SQ that are concurrent with O , O must have been inclusively transformed against them when we process $ET(O, SQ)$. For example, in Figure 1, to do $IT(O_1^{S_3^1}, O_2^{S_3^1})$, we perform $ET(O_1^{S_3^1}, O_3^{S_3^0})$ and $ET(O_2^{S_3^1}, O_3^{S_3^0})$ to compute the effect positions of O_1 and O_2 relative to S^0 . At this point, they both have inclusively transformed against O_3 . Hence we only need to record such a relation when processing ITs. Then ET can be correctly performed by retrieving the recorded effects relation. In this case it is not necessary to compute their effects relation again.

However, for any operation O_i in SQ such that $O_i \rightarrow O$, there is no recorded relation between them because we do not perform IT between causally related operations. And we cannot determine the $\prec$ relation between O_i and O by computing their β values relative to a LSP, because our concept of LSP is not defined between causally related operations. Then for this conceptual ET to work correctly, we have to use the position parameters of operations as far as we can. Let's examine the following three cases.

First consider the case in which O depends on O_i . We say that O_x depends on O_y iff $O_y \rightarrow O_x$ and O_x deletes the character inserted by O_y . As discussed in Section 4, relation $\prec$ is not defined between dependent operations. Thus we do not perform $ET(O, O_i)$ in this case. The IT/ET patch in Sun et al. (1998) is able to handle this situation but it loses the intuitive physical meaning of ET.

Second consider the case when processing $ET(O^{S^i}, O_i^{S^{i-1}})$, we have $O_i \rightarrow O, S^{i-1} + O_i^{S^{i-1}} = S^i$, and S^i is the generation state of O^{S^i} but O does not

depend on O_i . In this case, we can unambiguously determine the effects relation between O_i and O by their P parameters only, as defined in Algorithm 6. Note to distinguish the operation effects relation defined in Algorithm 3, here we annotate it as $\prec_{et}$.

Algorithm 6 $O_i^{S^{i-1}} \prec_{et} O^{S^i}$: *boolean*
 if $P(O_i^{S^{i-1}}) < P(O^{S^i})$ return true;
 else if $P(O_i^{S^{i-1}}) = P(O^{S^i})$
 if $T(O_i) = T(O) = \text{INS}$ return false;
 else if $T(O_i) = T(O) = \text{DEL}$ return true;
 else if $T(O_i) = \text{DEL}$ and $T(O) = \text{INS}$ return false;
 return false; //meaning $O^{S^i} \prec_{et} O_i^{S^{i-1}}$

The condition in Algorithm 6, $P(O_i^{S^{i-1}}) = P(O^{S^i})$, $T(O_i) = \text{DEL}$ and $T(O) = \text{INS}$, handles the case in which a character is deleted by O_i , and then immediately a character is inserted by O at the same position. In absence of semantic information, we assume at the system level either $C(O) \prec C(O_i)$ or $C(O_i) \prec C(O)$ is acceptable if the effect of O_i is excluded. Here we always enforce that $O \prec O_i$.

Third consider the case when processing $\text{ET}(O^{S^i}, O_i^{S^{i-1}})$, we have $O_i \rightarrow O$ but S^i is not the generation state of O . To illustrate, suppose three sites start from the same initial state $S^0 = \text{"abcd"}$. Site 1 performs $O_1 = \text{delete("c",2)}$ and concurrently site 2 performs $O_2 = \text{insert("x",3)}$. After site 3 receives and executes O_1 , its state becomes $S_3^1 = \text{"abd"}$. When O_2 arrives, because $O_2 \parallel O_1$, it is inclusively transformed against O_1 to be $O_2^{S_3^1} = \text{insert("x",2)}$. After $O_2^{S_3^1}$ is executed, the state becomes $S_3^2 = \text{"abxd"}$. Then site 3 performs a local operation, $O_3 = \text{insert("y",3)}$, which changes its state to $S_3^3 = \text{"abxyd"}$. It is clear that the effects relation should be $C(O_1) \prec C(O_2) \prec C(O_3)$.

Now the operation log of site 3 is $[O_1^{S_3^0}, O_2^{S_3^1}, O_3^{S_3^2}]$. To find the effect position of O_3 relative to S^0 , the intuitive way is to exclusively transform O_3 against $O_2^{S_3^1}$ and $O_1^{S_3^0}$ in order, as is shown in Figure 3. Let $O_3^{S_3^1} = \text{ET}(O_3^{S_3^2}, O_2^{S_3^1}) = \text{insert}$

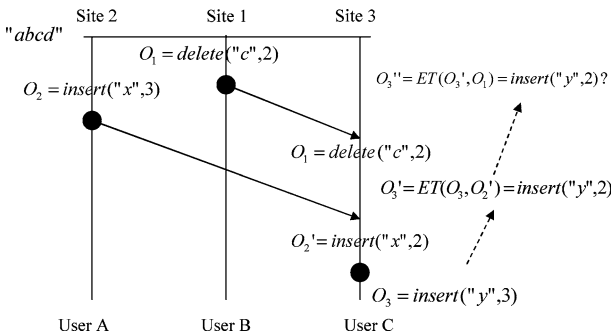


Figure 3. A puzzle scenario in ET between contextually serialized operations.

("y", 2). Then we calculate $O_3^{S_3^0} = \text{ET}(O_3^{S_3^1}, O_1^{S_3^0})$. The P values of these two operations are tied. The result would be $O_3^{S_3^0} = \text{insert}(\text{"y"}, 2)$, according to the ET functions defined in Sun et al. (1998). However, according to the above-identified effects relation, the correct result should be $O_3^{S_3^0} = \text{insert}(\text{"y"}, 3)$.

These three cases reveals the following important fact: Given two contextually serialized operations, $O_i^{S^{i-1}}$ and O^S , $O_i \rightarrow O$, and O does not depend on O_i , only when S^i is the generation state of O , is it safe to use position parameters to determine the effects relation, and are the original ET functions of Sun et al. (1998) equivalent to our conceptual definition of ET in Algorithm 5. Otherwise if S^i is not the generation state of O , we cannot always correctly determine their effects relation by directly using the ET functions of Sun et al. (1998), Sun (2002b).

This problem cannot be solved by simply recording the effects relation in IT. As revealed in Figure 3, due to $O_1 \rightarrow O_3$, O_3 was not inclusively transformed against O_1 before we process $\text{ET}(O_3^{S_3^1}, O_1^{S_3^0})$. Hence there is no recorded information to help remove the ambiguity. For similar reasons, the ET patch to solve the so-called lossy IT problems (Sun, 2002b) cannot solve this problem, either. This calls for extra measures in the control algorithm to prevent the problem from happening. Our method to correctly exclude the effects of contextually preceding operations will be presented in Section 7.

Besides, in our approach, we know $O_1 \prec O_2$ from $\text{IT}(O_2, O_1)$, and $O_2 \prec O_3$ from the observation that O_3 happened after O_2 and the effect position of O_3 is to the right of the character inserted by O_2 . Although it seems possible to transitively infer relation $O_1 \prec O_3$ such that the puzzle in Figure 3 is easily solved, we suspect that it is generally too expensive to infer relations such as $O_2 \prec O_3$ and $O_1 \prec O_3$ in practice. Hence currently these inferences only serve theoretical analysis purposes and we only record the effects relation between concurrent operations, as in Algorithm 12.

7. SD based transformation

In this section we address the problem of how to compute the effect (as characterized by β and δ) of any operation O relative to a state S^0 that is prior to its definition state S^n . Generally this is achieved by exclusively transforming O^{S^n} against the contextually serialized sequence SQ , the execution of which leads to state S^n . Due to our analysis in Section 6, our approach must safely avoid the problems of ET. The idea is based on a new concept called state difference. In the following, we introduce key concepts and related algorithms. These algorithms will be called by the control algorithm in Section 8.

7.1. STATE DIFFERENCE

The previous section defines the effect of a single operation relative to a given document state. Here we consider the effects of a sequence of operations. Because those operations may counteract each other, we introduce the concept of SD to denote their net effects. A SD is conceptually a compressed representation of an otherwise longer sequence of characterwise operations. It captures the increase and decrease of the document content.

Definition 5. (State difference) *Given a document state S^0 and a contextually serialized sequence of operations, $SQ = [O_1^{S^0}, \dots, O_n^{S^{n-1}}]$, such that $S^0 + SQ = S^n$ and $S^0 \leq S^n$. The state difference SD between S^0 and S^n is the net effects of all operations in SQ relative to S^0 , denoted as $S^0 + SD = S^n$. While the algorithm for building SD will be given later in Algorithm 8, a SD is conceptually obtained through the following three steps:*

(1) Computing operation effects

For every operation $O \in SQ$, compute its effect (β and δ values) relative to S^0 .

(2) Compressing operations

Many operations in SQ can be safely removed or combined. First, remove operation pairs with counteracting effects. Given any two characterwise operations, O_1 and O_2 , they counteract iff $O_1 \rightarrow O_2$, $T(O_1)=\text{INS}$, $T(O_2)=\text{DEL}$, and $\beta(O_1)=\beta(O_2)$, i.e., O_2 deletes the character inserted by O_1 . Second, combine all characterwise insert operations with the same β value into one stringwise insert operation, in which the characters in the string are ordered by their δ values. Note that delete operations are not combined, and insert operations at adjacent positions are merged regardless of their execution order in time.

(3) Reordering operations

For every operation in SD reset its position parameter to its β value. Now that SD is a set of operations whose position parameters are all based on S^0 , we reorder them according to their P values: Given any two operations O_i and O_j in SD , we make O_i precede O_j , iff (1) $P(O_i) < P(O_j)$, or (2) $P(O_i) = P(O_j)$ but $T(O_i)=\text{INS}$ and $T(O_j)=\text{DEL}$. This way SD is translated into a sort of sequence. However, it is no longer the contextually serialized sequence SQ whose operation order matches the original execution order.

7.2. SPECIAL OPERATION SEQUENCES

We define the following two special operation sequences as the basis for addressing the problems of ET. The negative order sequence will soon be used for computing β values in Algorithm 7.

Definition 6. (Net-effect operation sequence) *Given an operation sequence $SQ = [O_1, \dots, O_n]$, we say that SQ is a net-effect sequence, iff for any two operations in SQ , they cannot be compressed as in Definition 5.*

Definition 7. (Negative order operation sequence) *Given an operation sequence $SQ = [O_1, \dots, O_n]$ such that $S^0 + SQ = S^m$, we define the β value of every $O_i \in SQ$ on S^0 . We say that SQ is a negative order sequence, iff (1) SQ is a net operation sequence, (2) for any operation $O_i \in SQ$, $P(O_i) = \beta_{S^0}(O_i)$, and (3) for any two operations, $O_i, O_j \in SQ$ and $i < j$, we have (a) $P(O_i) > P(O_j)$, or (b) $P(O_i) = P(O_j)$ and $T(O_j) = INS$ and $T(O_i) = DEL$.*

Suppose SD is constructed from a contextually serialized sequence $SQ = [O_1^{S^0}, \dots, O_m^{S^{m-1}}]$ and $S^0 + SQ = S^m$. Let $SD = \{O_{x_1}^{S^0}, O_{x_2}^{S^0}, \dots, O_{x_n}^{S^0}\}$ and $S^0 + SD = S^m$, where $n \leq m$. By definition, a negative order sequence SQ_{ne} can be obtained by simply reversing the order of all operations in SD . In addition, operations SQ_{ne} can be understood again as a contextually serialized sequence: $SQ_{ne} = [O_{y_1}^{S^0}, O_{y_2}^{S^{1'}}, \dots, O_{y_n}^{S^{(n-1)'}}]$ and $S^0 + SQ_{ne} = S^0 + SD = S^m$, where sequence $y_1, \dots, y_n$ is a reverse of $x_1, \dots, x_n$, that is, $O_{y_i} = O_{x_{n+1-i}}$ for any $1 \leq i \leq n$. Note the superscripts in SQ_{ne} are primed and the subscripts in SD and SQ_{ne} are so written to distinguish from those in the original sequence SQ . The reason that SQ_{ne} is contextually serialized is due to the way its operations are ordered: For any pair of operations in SQ_{ne} , O_{y_i} and $O_{y_{i+1}}$, by definition we have $C(O_{y_{i+1}}) \prec C(O_{y_i})$. Hence the execution of O_{y_i} does not change the definition context of $O_{y_{i+1}}$. We can safely assume that $O_{y_{i+1}}$ is based on the state where O_{y_i} has just been executed.

7.3. COMPUTING OPERATION EFFECTS

When an operation O is executed on state S_i , its position is based on S_i . To decide its effect relative to an earlier state S_j , we must exclusively transform O against all the operations that have been executed since S_j and that have impacts on it. This problem can be formalized as follows: Given state S^0 , there exists a sequence of contextually serialized operations $SQ = [O_1^{S^0}, \dots, O_m^{S^{m-1}}]$ such that $S^0 + SQ = S^m$, and operation O^{S^m} is executed on S^m . We consider how to compute the effect of O^{S^m} relative to S^0 . Rather than processing $ET(O^{S^m}, SQ)$ directly, we build a state difference SD from SQ first and then compute $ET(O^{S^m}, SD)$ instead. The procedure is given in Algorithm 7.

Algorithm 7. Compute $\beta\delta(O, SD)$: $\langle \beta(O), \delta(O) \rangle$, where
 O is an operation; SD is a state difference; $\beta(O)$ is initialized as ϕ ;

```

1. rebuild a negative order operation sequence  $SQ_{ne}$  from  $SD$ ;
2. for  $(i=n; i \geq 1; i--)$  //  $n$  is the #members of  $SQ_{ne}$ 
3.   if  $P(O) < P(SQ_{ne}[i])$  then
4.      $\beta(O) := P(O)$ ;  $\delta(O) := 0$ ; end loop;
5.   elseif  $T(SQ_{ne}[i]) = \text{INS}$  and  $T(O) = \text{INS}$  and  $P(O) = P(SQ_{ne}[i]) + L(SQ_{ne}[i])$  then
6.     if  $T(SQ_{ne}[i-1]) = \text{DEL}$  and  $P(SQ_{ne}[i]) = P(SQ_{ne}[i-1])$  then
7.        $O_t := \text{ET}(O, SQ_{ne}[i], SQ_{ne}[i-1])$ ;
8.       if  $P(O_t) > P(O)$  then
9.          $P(O) := P(O_t)$ ;
10.         $i := i-1$ ; //skip  $SQ_{ne}[i-1]$ ;
11.      else //  $P(O)$  does not change
12.         $\beta(O) := P(SQ_{ne}[i])$ ;  $\delta(O) := P(O) - P(SQ_{ne}[i])$ ;
13.      end loop;
14.    endif
15.  else //not DEL or delete at another position
16.     $\beta(O) := P(SQ_{ne}[i])$ ;  $\delta(O) := P(O) - P(SQ_{ne}[i])$ ;
17.    end loop;
18.  endif
19.  elseif  $T(SQ_{ne}[i]) = \text{INS}$  and  $P(SQ_{ne}[i]) \leq P(O) < P(SQ_{ne}[i]) + L(SQ_{ne}[i])$  then
20.     $\beta(O) := P(SQ_{ne}[i])$ ;
21.    if  $T(O) = \text{INS}$  then  $\delta(O) := P(O) - P(SQ_{ne}[i])$ ;
22.    if  $T(O) = \text{DEL}$  then  $\delta(O) := P(O) - P(SQ_{ne}[i]) + 1$ ;
23.    end loop;
24.  else  $O := \text{ET}(O, SQ_{ne}[i])$ ;
25.  endif
26. endfor
27. if  $\beta(O) = \phi$  then  $\beta(O) := P(O)$ ;  $\delta(O) := 0$ ;
28. return  $\langle \beta(O), \delta(O) \rangle$ .
```

Suppose $SD = \{O_{x_1}^{S^0}, O_{x_2}^{S^0}, \dots, O_{x_n}^{S^0}\}$, where $P(O_{x_1}^{S^0}) \leq P(O_{x_2}^{S^0}) \leq \dots \leq P(O_{x_n}^{S^0})$ and $n \leq m$. We first rebuild a negative order sequence SQ_{ne} from SD by reversing the order of all operations, that is, $SQ_{ne} = [O_{y_1}^{S^0}, O_{y_2}^{S^0}, \dots, O_{y_n}^{S^0}]$, where $O_{y_i} = O_{x_{n+1-i}}$. Intuitively SQ_{ne} is a contextually serialized sequence of operations that effectively generate the same effects (as characterized by SD) on S^0 as the original SQ . Thus switching from SQ to SQ_{ne} does not change the execution context of operation O^{S^m} . That is, $S^0 + SQ_{ne} = S^0 + SQ = S^m$. However, this rebuilt sequence SQ_{ne} holds properties that can help remove the defect of ET when O^{S^m} is exclusively transformed against SQ_{ne} , as will be proved later.

Then we scan SQ_{ne} from right to left and exclusively transform O^{S^m} against $SQ_{ne}[i]$ in order as necessary. The reason follows. When SQ_{ne} is rebuilt from SD , we already *assume* that operations in SQ_{ne} are contextually serialized. Then according to the original definition of ET in Sun et al. (1998), ET must be performed from right to left.

We consider the following three cases. The first (lines 3–4) is straightforward: When O is compared with the first $O_{y_i}^{S^{(i-1)'}}$ where $P(O) < P(O_{y_i}^{S^{(i-1)'}})$, it has already been transformed against the subsequence of $SQ_{ne}[i+1, n]$. It is no longer necessary to transform it against $SQ_{ne}[1, i]$. Because $\forall O_r \in SQ_{ne}[1, i-1], O_{y_i} \prec O_r$ and $O \prec O_{y_i}$, we have $O \prec O_r$. Therefore performing $ET(O, SQ_{ne}[1, i])$ does not really shift the position parameter of O .

In the second case (lines 19–23), O depends on $SQ_{ne}[i]$. For example, let $SQ_{ne}[i] = \text{insert}(\text{"xy"}, 2)$. Then O could be to insert a new character into string "xy" or to delete a character from it. Either way $\beta(O)$ is set to the effect position of $SQ_{ne}[i]$ relative to S^0 and $\delta(O)$ is set accordingly. After we find the β and δ values of O , it is no longer necessary to examine $SQ_{ne}[1, i-1]$. As in the first case, we break the loop. Note here although it is not accurate to say that $SQ_{ne}[i] \rightarrow O$ because $SQ_{ne}[i]$ is an artificial operation, O should depend on at least one of the source operations from which it is derived. Also note that in this case we do not perform $ET(O, SQ_{ne}[i])$ to calculate $\beta(O)$ and $\delta(O)$.

The third case (lines 5–18) is more complicated. The condition under which this case arises is given in line 5. This is illustrated by the following example. Assume $S^0 = \text{"abcd"}$ is the initial state, $SQ_{ne} = [O_1^{S^0}, O_2^{S^1}]$, $O_2^{S^1} = \text{insert}(\text{"xy"}, 2)$, and $S^0 + SQ_{ne} = S^2$. Now $O^{S^2} = \text{insert}(\text{"z"}, 4)$ is executed on S^2 . Then what is the position of O^{S^2} relative to S^0 ? There are two possibilities: the character 'z' is inserted either immediately before or after the landmark character 'c'. In the first possibility, $\beta_{S^0}(O) = 2$ and $\delta_{S^0}(O) = 2$, while in the second, $\beta_{S^0}(O) = 3$ and $\delta_{S^0}(O) = 0$. The result depends on the operation $O_1^{S^0}$.

It is not possible to compute $P(O^{S^0})$ only by comparing O^{S^2} and $O_2^{S^1}$. To find the answer, we have to check the next element in SQ_{ne} , $O_1^{S^0}$. If $O_1^{S^0}$ is an insert operation, then $P(O_1^{S^0}) > P(O_2^{S^1})$ must hold. If O_1 is delete operation but $P(O_1^{S^0}) \neq P(O_2^{S^1})$, then $P(O_1^{S^0}) > P(O_2^{S^1})$ must hold. Either way (lines 15–17) $O_1^{S^0}$ does not impact $P(O^{S^0})$, resulting that 'z' is inserted immediately before 'c'.

If $O_1^{S^0} = \text{delete}(\text{"c"}, 2)$, we need to find the inserted position of 'z' relative to S^0 by processing $ET(ET(O^{S^2}, O_2^{S^1}), O_1^{S^0})$, as shown in lines 6–14. It is clear that $O^{S^1} = \text{insert}(\text{"z"}, 2)$. What is the result of $O^{S^0} = ET(O^{S^1}, O_1^{S^0})$? There are two possibilities. On one hand, if $O \parallel O_1$, then O must have been inclusively transformed against O_1 . The result of ET can then be unambiguously computed by checking the recorded effects relation between O_1 and O . On the other hand, if $O_1 \rightarrow O$, since $P(O^{S^1}) = P(O_1^{S^0})$, $T(O_1) = \text{DEL}$ and $T(O) = \text{INS}$, the result of ET is not unique. Note here $T(O_1) = \text{DEL}$ and, according to the construction process of SD , delete operations that appear in SD are always character-based. O^{S^0} could be $\text{insert}(\text{"z"}, 2)$ or $\text{insert}(\text{"z"}, 3)$. However, since 'z' is inserted by the user after 'c' has been deleted, either result is acceptable at the system level. Here we always do not shift the position parameter of O so that one is deterministically decided. Hence the ambiguity can be resolved by Algorithms 5 and 6.

Depending on the outcome of O^{S^0} , either lines 8–10 or 11–13 will be processed. Note here both O_2 and O are insert operations. If O^{S^0} turns out to insert a character at the same position relative to S^0 as O_2 , as is the case of lines 11–13, then $C(O)$ should be merged into $S(O_2)$. Otherwise (lines 8–10), O^{S^0} is to insert a character after $C(O_1)$, assuming that $C(O_1)$ would not have been deleted by O_1 . Then $P(O)$ is set to that of the ET result and the exclusion transformation loop continues if there are still more operations in SQ_{ne} to consider. However, the lookahead operation O_1 should be skipped since its effect has already been excluded.

7.4. BUILDING SD

Assume a sequence $SQ = [O_1^{S^0}, \dots, O_n^{S^{n-1}}]$ is executed on S^0 and $S^0 + SQ = S^n$. We can build a state difference SD from SQ incrementally, as shown in Algorithm 8.

Algorithm 8. *Build $SD(SQ)$: SD , where SQ is a sequence of n operations.*

```

 $SD := \{SQ[1]\};$ 
for( $i = 2$ ;  $i \leq n$ ;  $i++$ )
     $\langle \beta_{S^0}(SQ[i]), \delta_{S^0}(SQ[i]) \rangle := \text{Compute}\beta\delta(SQ[i], SD);$ 
    merge  $SQ[i]$  into  $SD$ ;
return  $SD$ ;

```

We first initialize SD to $SQ[1]$. Then we incrementally incorporate the effects of other operations in SQ into the partially built SD . Let's consider how an operation $O_i^{S^{i-1}}$ is incorporated. First we calculate the effect of $O_i^{S^{i-1}}$ relative to S^0 by calling algorithm $\text{Compute}\beta\delta$. Then we merge O_i into SD according to its β and δ values. Due to the definition of state difference, we only need to consider two cases:

- (1) if $\beta(O_i)$ is not the same as the β value of any existing insert operation in SD , then simply insert O_i into SD in position order.
- (2) otherwise locate the insert operation O_j in SD such that $P(O_j) = \beta_{S^0}(O_i)$. Since $S(O_j)$ is a string, insert $C(O_i)$ into $S(O_j)$ if $T(O_i) = \text{INS}$, or delete $C(O_i)$ from $S(O_j)$ if $T(O_i) = \text{DEL}$, according to the value of $\delta_{S^0}(O_i)$.

7.5. TIMESTAMPING OPERATIONS IN SD

Every delete operation in a negative order sequence (SQ_{ne}) needs to carry a timestamp for determining the partial ordering relation between it and other operations. As discussed in Section 6, when exclusively transforming an insertion against a deletion, if they have the same position parameter, the result may be ambiguous. However, if they are concurrent, we can always obtain the correct result by checking the stored effects relation between them.

Since a deletion in SQ_{ne} always corresponds to an operation in the original sequence SQ , its timestamp remains.

Insert operations in SQ_{ne} , on the other hand, do not need to carry timestamps. When exclusively transforming any operation against an insertion, we can always get the right result by only comparing their position parameters.

8. The integration procedure

The top-level control algorithm works as follows. Every time when a causally ready operation O is to be executed, the integration algorithm is called to compute O' , the executable form of O based on the current state. If O' is not an identity operation, it is executed in current state and then appended to the history buffer HB , which stores all locally executed operations. The following Algorithms 9–11 are similar to the control algorithm of SOCT2 (Suleiman et al., 1997, 1998) and GOTO (Sun and Ellis, 1998). They are included for this paper to be self-contained.

Algorithm 9. Integrate (O, HB_c): O' , where O is a causally ready operation; HB_c is a copy of HB , a sequence of n operations; and O' is a transformed version of O that is executable on current document state.

1. for $i = 1$ to n do
2. if $HB_c[i] \parallel O$ //the first concurrent operation
3. break;
4. if $i > n$ //no concurrent operation
5. return $O' := O$;
6. $\langle SQ_{i,p} \rangle := \text{TransposeR2L}(O, HB_c[i,n])$;
7. $HB_c := HB_c[1, i-1] + SQ_i$;
8. $k := (i-1) + p$; //point to the first concurrent operation in HB_c
9. $O' := \text{SQIT}(O, HB_c, k)$;
10. return O' ;

Algorithm 9 gives the integration procedure. For a causally ready operation O , we first scan HB_c (a copy of HB) from left to right until the first operation that is concurrent with O is found (lines 1–3). Assume this is the i th operation in HB_c , denoted as $HB_c[i]$. If there is no such operation, then O can be directly applied on the current state (lines 4–5). Otherwise, we need to re-organize $HB_c[i,n]$ by calling TransposeR2L so that all operations that causally precede O are transposed before those operations that are concurrent with O (lines 6–7). After that, we get an updated HB_c , in which the k th operation is the first that is concurrent with O . Then we inclusively transform O against the subsequence $HB_c[k,n]$, all operations in which are concurrent with O , by calling SQIT (lines 8–9).

8.1. TRANSPOSING SEQUENCE FROM RIGHT TO LEFT

Algorithm 10. *TransposeR2L* (O, SQ) : $\langle SQ_r, p \rangle$, where O is the given remote operation; SQ is a sequence of n operations; SQ_r is a resultant operation sequence; and p is the position of the leftmost operation concurrent with O in SQ_r .

```

1.  $SQ_r := SQ$ ;
2.  $p := 1$ ;
3. for  $i = 1$  to  $n$  do
4.   if  $SQ_r[i] \rightarrow O$ 
5.      $SQ_i := R2LIT(SQ_r[p, i])$ ;
6.      $SQ_r := SQ_r[1, p-1] + SQ_i + SQ_r[i+1, n]$ ;
7.      $p := p + 1$ ;
8.   endif
9. return  $\langle SQ_r, p \rangle$ ;
```

The purpose of the *TransposeR2L* algorithm is to change the operation order of a given contextually serialized operation sequence. The given remote operation O is used to identify operations in SQ that causally precede O . Concretely, supposing $SQ[1] \parallel O$, each time an operation $SQ[i] \rightarrow O$ is found, algorithm *R2LIT* is called to transpose $SQ[i]$ to the position pointed to by p . Initially $SQ[p] \parallel O$, and p is adjusted to point to the leftmost operation concurrent with O . As a result, all operations that causally precede O are transposed to the left of those that are concurrent with O .

Algorithm 11. *R2LIT* (SQ): SQ' , where SQ is a sequence of n operations; and SQ' is the transposed sequence.

```

1.  $SQ' := SQ$ ;
2.  $O_{i+1} := SQ'[n]$ ;
3. for  $i = n-1$  to 1 do
4.    $O_{i+1} := ET(O_{i+1}, SQ'[i])$ ;
5.    $O_i := IT(SQ'[i], O_{i+1})$ ;
6.    $SQ' := SQ'[1, i-1] + [O_{i+1}, O_i] + SQ'[i+2, n]$ ;
7. return  $SQ'$ ;
```

Given an operation sequence SQ with n operations, algorithm *R2LIT* transposes its rightmost operation $SQ[n]$ to the leftmost position. For the above-given operation O , it is assumed that only $SQ[n] \rightarrow O$ and all operations in $SQ[1, n-1] \parallel O$. In the loop of lines 3–6 we transpose $SQ[n]$ one position to the left at each step. To transpose $SQ[i]$ and $SQ[i+1]$, for example, we first compute $SQ[i+1]' = ET(SQ[i+1], SQ[i])$. Then this result is used to get $SQ[i]' = IT(SQ[i], SQ[i+1]')$. After that, sequence SQ is modified such that the positions of $SQ[i]'$ and $SQ[i+1]'$ are exchanged. This procedure is incrementally processed until $SQ[n]$ is moved to the leftmost side of SQ .

Now we explain why *ET* and *IT* can be correctly processed here (lines 4–5). It is assumed that $O_{i+1} \rightarrow O$ and $O_i \parallel O$. Hence it is impossible to have $O_i \rightarrow O_{i+1}$, otherwise, due to transitivity of causal ordering, we have

$O_i \rightarrow O$, which contradicts $O_i \parallel O$. It is also impossible to have $O_{i+1} \rightarrow O_i$, otherwise, due to causality preservation, O_{i+1} should have been executed before O_i . Therefore the relation between O_i and O_{i+1} must be $O_i \parallel O_{i+1}$. Between these two concurrent operations, their effects relation has been stored when $IT(O_{i+1}, O_i)$ was performed in order to execute O_{i+1} . Hence there is no ambiguity here in processing IT between them again. ET between them can also be correctly processed according to the stored effects relation.

8.2. TRANSFORMING CONCURRENT OPERATIONS

Algorithm 12. $SQIT(O, SQ, p)$: O' , where O is a remote operation; SQ is an operation sequence of n operations; p is an index such that all operations in $SQ[p, n]$ are concurrent with O ; and O' is the transformed version of O .

1. for $i = p$ to n do
2. $SQ_{lsp} := \text{ComputeLSP}(O, i, SQ)$;
3. $SD := \text{BuildSD}(SQ_{lsp})$;
4. $\langle \beta_1, \delta_1 \rangle := \text{Compute}\beta\delta(O, SD)$;
5. $\langle \beta_2, \delta_2 \rangle := \text{Compute}\beta\delta(SQ[i], SD)$;
6. $O := IT(O, SQ[i])$;
7. Store the effects relation between O and $SQ[i]$;
8. return $O' := O$;

Algorithm SQIT inclusively transforms a given operation O against all operations in SQ that are current with O . For the given index p , it is assumed that all operations in subsequence $SQ[1, p-1]$ causally precedes O and those in $SQ[p, n]$ are concurrent with O . For each $SQ[i] \parallel O$, we first call algorithm ComputeLSP to get their LSP. Then we build a SD based on LSP. After that, the operation effects (β and δ values) of $SQ[i]$ and O relative to LSP are computed by calling algorithm $\text{Compute}\beta\delta$ (Algorithm 7), respectively. Then we perform $IT(O, SQ[i])$ as discussed earlier in Section 5. Finally the effects relation between these two operations is stored in case of future use.

Algorithm 13. $\text{ComputeLSP}(O, j, SQ)$: SQ_{lsp} , where O is a remote operation ready to be synchronized; j is the index such that $SQ[j]$ is to be transformed with O ; SQ is the given operation sequence with n operations; and SQ_{lsp} is the result operation sequence.

1. $SV_{lsp} := \text{MIN}(V(O), V(SQ[j]))$;
2. if $\exists r((V(SQ[r]) \leq SV_{lsp}) \text{ and } (\forall O_s(O_s \in SQ[r+1, n] \Rightarrow V(O_s) \not\leq SV_{lsp}))$
3. $\langle SQ_t, i \rangle := \text{TransposeL2R}(SV_{lsp}, SQ[1, r])$;
4. $SQ_t := SQ_t + SQ[r+1, n]$;
5. $SQ_{lsp} := SQ_t[i+1, j-1]$;
6. else //timestamps of all operations in $SQ[1, n]$ are no less than SV_{lsp}
7. $SQ_{lsp} := SQ[1, j-1]$;
8. endif;
9. return SQ_{lsp} ;

Now we examine algorithm ComputeLSP. Operation O is to be inclusively transformed against the operation in SQ that is pointed to by index j , i.e., $SQ[j]$. As shown in line 1, we first compute SV_{lsp} , the state vector identifying the LSP of O and $SQ[j]$, by calling function MIN to get the minimum value of each corresponding element of the vector timestamps of O and $SQ[j]$.

Then we find the subsequence in SQ that can be used to build the desired SD. As denoted in line 2, we scan SQ from right to left to find the first operation $SQ[r]$ such that its timestamp is less than or equal to SV_{lsp} and the timestamps of all operations in $SQ[r+1, n]$ are not less than or equal to SV_{lsp} . If $SQ[r]$ exists, we must have $SQ[r] \rightarrow SQ[j]$ and $r < j$ due to causality preservation. If such an operation is not found (lines 6–7), the timestamps of all operations in $SQ[1, n]$ must be not less than or equal to SV_{lsp} , which implies that LSP is the initial state. Hence we return subsequence $SQ[1, j-1]$.

If r is found (lines 2–5), there may still be operations in the subsequence $SQ[1, r]$ whose timestamps are not less than or equal to SV_{lsp} . Hence we need to transpose $SQ[1, r]$ such that all operations whose timestamps are less than or equal to SV_{lsp} are moved to the left of those whose timestamps are not less than or equal to SV_{lsp} . Let i point to the rightmost operation in the transposed sequence whose timestamp is less than or equal to SV_{lsp} . After concatenating the transposed sequence and $SQ[r+1, n]$, the new sequence has the property that the timestamps of all operations in $SQ[i, n]$ are not less than or equal to SV_{lsp} . In this case, subsequence $SQ[i+1, j-1]$ is what we want.

Due to algorithm TransposeR2L, we have $SQ[k] \rightarrow O$ for any operation $SQ[k] \in SQ[1, r]$. Those operations whose timestamps are less than or equal to SV_{lsp} must have contributed to S^{lsp} , while those with timestamps not less than or equal to SV_{lsp} have not. Algorithm TransposeL2R transposes $SQ[1, r]$ such that the execution of operations in the left part led to S^{lsp} while that of the right-side operations does not have anything to do with it. Then the SD is built from the right-side operations up to $SQ[j-1]$.

Algorithm 14. *TransposeL2R(SV, SQ) : $\langle SQ_r, r \rangle$, where SV is a given state vector; SQ is a given operation sequence of length n ; SQ_r is the transposed result sequence; and r points to the rightmost operation in SQ_r whose vector timestamp is less than or equal to SV .*

1. $SQ_r = SQ$;
2. $r := n$;
3. for $i = n-1$ to 1 do
4. if $V(SQ_r[i]) \not\leq SV$
5. $SQ_i := \text{L2RIT}(SQ_r[i, r])$;
6. $SQ_r := SQ_r[1, i-1] + SQ_i + SQ_r[r+1, n]$;
7. $r := r-1$;
8. endif;
10. return $\langle SQ_r, r \rangle$;

8.3. TRANSPOSING SEQUENCE FROM LEFT TO RIGHT

Algorithm TransposeL2R transposes a given contextually serialized operation sequence such that all operations whose timestamps are less than or equal to SV are moved to the left of those whose timestamps are not less than or equal to SV . Note it is assumed in the beginning of this algorithm that the rightmost operation in SQ here has a timestamp less than or equal to SV , according to algorithm ComputeLSP. Assume that SQ has n operations. We scan SQ from right to left, every time an operation $SQ[i]$ with a timestamp not less than or equal to SV is found, we call algorithm L2RIT to transpose it to the rightmost position of sequence $SQ[i, r]$. Index r is maintained such that it always points to the rightmost operation with a vector timestamp less than or equal to SV . Algorithm 15. *L2RIT(SQ): SQ' , where SQ is a given sequence of n operations; and SQ' is the transposed sequence.*

1. $SQ' := SQ$;
2. $O_{i-1} := SQ'[1]$;
3. for $i = 2$ to n do
4. $O_i := ET(SQ'[i], O_{i-1})$;
5. $O_{i-1} := IT(O_{i-1}, O_i)$;
6. $SQ' := SQ'[1, i-2] + [O_i, O_{i-1}] + SQ'[i+1, n]$;
7. return SQ' ;

Given an operation sequence SQ of length n , algorithm L2RIT is responsible for transposing $SQ[1]$ to the rightmost side of SQ . Based on algorithm TransposeL2R, we can assume here that only the timestamp of $SQ[1]$ is not less than or equal to SV and that the timestamps of all operations in $SQ[2, n]$ are less than or equal to SV . In the loop of lines 3–6, at each step we transpose O_{i-1} and O_i in a way similar to algorithm R2LIT. Similarly those operations are concurrent and IT/ET can be correctly processed here.

9. An example

A simple example is given in Figure 4. Three users start from the same initial state $S^0 = \text{"abcd"}$ at all sites. User A performs $O_1 = \text{insert("2", 2)}$ at site 1. Concurrently user B performs $O_2 = \text{delete("b", 1)}$ at site 2, user C performs $O_3 = \text{insert("1", 1)}$ at site 3. O_2 and O_3 are received and executed at site 1 in order. Let $S_1^0 + [O_1] = S_1^1 = \text{"ab2cd"}$, $S_1^1 + [O_2^{S_1^1}] = S_1^2 = \text{"a2cd"}$, and $S_1^2 + [O_3^{S_1^2}] = S_1^3 = \text{"a12cd"}$, where $O_2^{S_1^1} = IT(O_2, O_1) = \text{delete("b", 1)}$, and $O_3^{S_1^2} = IT(O_3, [O_1, O_2^{S_1^1}]) = \text{insert("1", 1)}$. Now user A performs $O_4 = \text{insert("4", 3)}$ and the state becomes $S_1^3 = \text{"a124cd"}$. After that user A receives a concurrent operation $O_5 = \text{insert("3", 3)}$ performed by user C. Now the operation log of site 1 is $[O_1^{S_1^0}, O_2^{S_1^1}, O_3^{S_1^2}, O_4^{S_1^3}]$. It is interesting here to examine how operation O_5 is integrated.

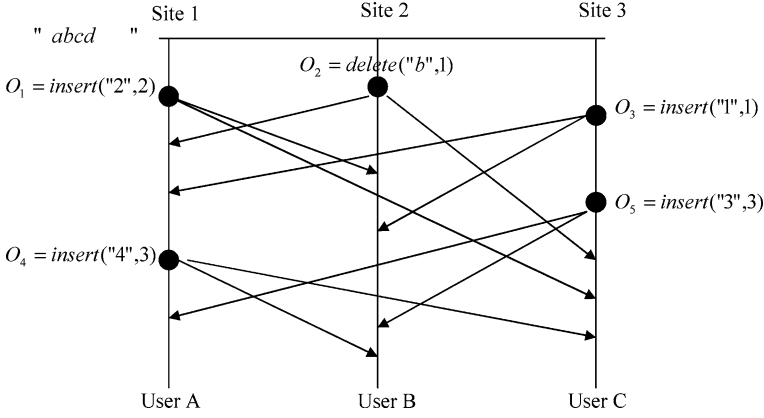


Figure 4. An example including typical OT puzzles.

Among the four operations in the log, the following relations are observed: $O_1 \parallel O_5, O_2 \parallel O_5, O_3 \rightarrow O_5$ and $O_4 \parallel O_5$. Following the control algorithm discussed above, O_1 and O_2 are transposed with O_3 and the log becomes $[O_3^{S_1^0}, O_1^{S_1^{1'}}, O_2^{S_1^{2'}}, O_4^{S_1^3}]$, where $O_1^{S_1^{1'}} = \text{insert}('2', 3)$ and $O_2^{S_1^{2'}} = \text{delete}('b', 2)$. The execution form of O_4 remains because its execution state is not changed by the transposition.

Then we need to inclusively transform O_5 against $O_1^{S_1^{1'}}, O_2^{S_1^{2'}}$ and O_4 in order. Strictly O_5 should be written as $O_5^{S_1^3}$ where $S_1^3 = S_1^0 + [O_3]$. Note S_1^3 and $S_1^{1'}$ are equivalent because they are both obtained by executing the same operation O_3 on the same state. Hence O_5 can also be written as $O_5^{S_1^{1'}}$. After inclusively transforming O_5 against $O_1^{S_1^{1'}}, O_2^{S_1^{2'}}$ we get $O_5^{S_1^3} = \text{insert}('3', 3)$. Now we closely examine how $O_5^{S_1^3}$ is inclusively transformed against $O_4^{S_1^3}$.

We first compute the beta values of O_4, O_5 in their LSP, $\beta_{45}(O_4)$ and $\beta_{45}(O_5)$, respectively. The vector timestamps of O_4 and O_5 are $V(O_4) = (2, 1, 1)$ and $V(O_5) = (0, 0, 2)$. Thus $SV_{lsp} = (0, 0, 1)$ and the LSP of O_4 and O_5 is $S_1^{1'} = \text{"a1bcd"}$, which is the result of only executing O_3 on the initial state S_1^0 . Hence $\beta_{45}(O_5) = 3$, because O_5 is generated on this state.

Now we calculate $\beta_{45}(O_4)$. The operation sequence between the LSP and the definition state of O_4 is $[O_1^{S_1^{1'}}, O_2^{S_1^{2'}}]$. Hence the SD is $SD = \{\text{delete}('b', 2), \text{insert}('2', 3)\}$. Due to the $\text{Compute}\beta\delta(O_4, SD)$ algorithm, we have $\beta_{45}(O_4) = 3$. Since $\beta_{45}(O_4) = \beta_{45}(O_5)$ and $P(O_4^{S_1^3}) = P(O_5^{S_1^3})$, we must resort to comparing their site ids to determine their effects relation. As a result, we have $O_5^{S_1^3} = \text{insert}('3', 4)$, where $S_1^4 = S_1^3 + [O_4]$. After executing $O_5^{S_1^3}$, the state of site 1 becomes $S_1^5 = \text{"a1243cd"}$.

At site 2, after the execution of O_2 the state becomes $S_2^1 = \text{"acd"}$. When O_1 arrives, we have $O_1^{S_2^1} = \text{IT}(O_1, O_2) = \text{insert}('2', 1)$. When O_3 arrives, it is

first transformed against O_2 , which results in $O_3^{S_2^1} = \text{insert}("1", 1)$. Then it is interesting to examining how to compute $O_3^{S_2^2} = \text{IT}(O_3^{S_2^1}, O_1^{S_2^1})$, where $S_2^1 + [O_1^{S_2^1}] = S_2^2 = \text{"a2cd"}$. Obviously their LSP is $S_2^0 = \text{"abcd"}$, which is the generation state of both O_1 and O_3 . Hence we have $\beta_{13}(O_1) = 2$ and $\beta_{13}(O_3) = 1$. Since $\beta_{13}(O_1) > \beta_{13}(O_3)$, we have $O_3^{S_2^2} = \text{insert}("1", 1)$, the execution of which results in $S_2^3 = \text{"a12cd"}$.

When O_5 arrives at site 2, we first transpose the log $[O_2^{S_2^0}, O_1^{S_2^1}, O_3^{S_2^2}]$ to $[O_3^{S_2^0}, O_1^{S_2^{1'}}, O_2^{S_2^{2'}}]$, where $S_2^0 + O_3^{S_2^0} = S_2^{1'}$ and $S_2^{1'} + O_1^{S_2^{1'}} = S_2^{2'}$. Then we transform O_5 against $O_1^{S_2^{1'}}$ and $O_2^{S_2^{2'}}$ in order and get $O_5^{S_2^3} = \text{insert}("3", 3)$, where $S_2^{2'} + [O_2^{S_2^{2'}}] = S_2^3$. Finally, O_4 arrives at site 2. Similarly we get $O_4^{S_2^4} = \text{insert}("4", 3)$, where $S_2^3 + [O_4^{S_2^3}] = S_2^4$. After $O_4^{S_2^4}$ is executed, the state of site 2 is $S_2^5 = \text{"a1243cd"}$.

At site 3, after the execution of O_3 and O_5 , the state becomes $S_3^2 = \text{"a1b3cd"}$. Then O_2 , O_1 , and O_4 arrive in order. We have $O_2^{S_3^2} = \text{delete}("b", 2)$, $O_1^{S_3^3} = \text{insert}("2", 2)$, and $O_4^{S_3^4} = \text{insert}("4", 3)$, where $S_3^2 + [O_2^{S_3^2}] = S_3^3$, $S_3^3 + [O_1^{S_3^3}] = S_3^4$. After the execution of all operations, the state of site 3 is $S_3^5 = \text{"a1243cd"}$.

It is easy to see that our approach correctly preserves the operation effects relation and the final states converge after all operations are executed at all sites.

10. Correctness proof

Our control algorithm (specifically TransposeR2L) follows that of GOTO (Sun and Ellis, 1998). It is understood that this reordering of history buffer before the integration of remote operations can solve the well-known dOPT puzzle (Sun and Ellis, 1998). In the subsections that follow, we prove the following aspects of our approach, respectively, (1) the correctness of ET (which is used as a basis of computing β and δ) due to the use of negative order sequence, (2) the correctness of computing β and δ as a basis of processing IT, (3) the satisfaction of TP1 and TP2 in our IT functions (and thus convergence), and (4) the preservation of single-operation effects and multi-operation effects relation. As a corollary, our approach ensures CSM consistency. It always preserves operation effects relation and is thus able to guarantee convergence regardless of transformation paths.

10.1. CORRECTNESS OF ET

In Section 6, we have discussed the defect of existing ET functions as defined in Sun et al. (1998). In fact, this defect complicates the computing of β and δ values of an operation relative to an earlier document state. To

fix this problem, we introduced the concept of negative order operation sequence. This special sequence can ensure that for two operations $O_1^{S^0}$ and $O_2^{S^1}$, even if S^1 is not the generation state of O_2 , the ET functions can still correctly determine their effects relation. That is, $ET(O_2^{S^1}, O_1^{S^0})$ always gets the correct result. We first give a lemma, based on which it will be easy to prove the theorem.

Lemma 1. *Given three contextually serialized operations, $O_1^{S^0}$, $O_2^{S^1}$ and $O_3^{S^2}$, where (1) $S^0 + [O_1^{S^0}] = S^1$, $S^1 + [O_2^{S^1}] = S^2$, (2) $C(O_2) \prec C(O_1)$, and (3) O_3 depends on neither O_1 nor O_2 . If $ET(O_3^{S^2}, O_2^{S^1})$ is correct, then $ET(O_3^{S^1}, O_1^{S^0})$ is correct.*

Proof. Based on the position of $C(O_3)$ relative to $C(O_2)$ and $C(O_1)$, the problem is divided into three cases: (1) $C(O_3) \prec C(O_2) \prec C(O_1)$, (2) $C(O_2) \prec C(O_3) \prec C(O_1)$, and (3) $C(O_2) \prec C(O_1) \prec C(O_3)$. The procedure of proving either case is similar. Hence we only choose to prove case (2), which is the most complicated.

To help understand the problem, we assume that the character deleted by any operation O is not really removed. Instead, it is only marked with a $\star$ in the internal data structure. For example, if $T(O_1) = \text{DEL}$ and $T(O_2) = \text{INS}$, then S^2 will look like "...C (O_2), ..., C (O_1) $\star$, ...". Let P_i^j denote the position parameter of O_i in state S^j . Then P_1^2 and P_2^2 respectively point to the position of $C(O_1)$ and $C(O_2)$ in state S^2 . Since the deleted character $C(O_1)$ does not really take any position in S^2 , it is possible that $P_2^2 = P_1^2$. We consider the following four cases.

(1) $T(O_1) = \text{DEL}$, $T(O_3) = \text{DEL}$ and $T(O_2) = \text{DEL}$. Due to the given precondition, $C(O_2) \prec C(O_3) \prec C(O_1)$, state S^2 (before the execution of $O_3^{S^2}$) should look like "...C (O_2) $\star$, ..., C (O_3) $\star$, ..., C (O_1) $\star$, ...". Since the position of $C(O_2)$ $\star$ and $C(O_1)$ $\star$ do not really count, we have $P_2^2 \leq P_3^2 \leq P_1^2$. Consider P_1^1 and P_3^1 , which respectively represents the position of $C(O_1)$ and $C(O_3)$ in state S^1 . Due to our assumption, $ET(O_3^{S^2}, O_2^{S^1})$ can be correctly processed. That is, we can correctly get the position value of O_3 relative to S^1 , $P_3^1 = P_3^2 + 1$. Similarly, once $C(O_2)$ is put back in S^1 , the position of $C(O_1)$ relative to S^1 becomes $P_1^1 = P_1^2 + 1$. Hence $P_3^1 < P_1^1$. As a result, excluding the effect of O_1 does not affect the value of P_3 . Therefore relation $O_3 \prec O_1$ maintains and $ET(O_3^{S^1}, O_1^{S^0})$ can be correctly processed.

(2) $T(O_1) = \text{DEL}$, $T(O_3) = \text{DEL}$ and $T(O_2) = \text{INS}$. Due to the given precondition, we have $P_2^2 P_3^2 P_1^2$. After correctly processing $ET(O_3^{S^2}, O_2^{S^1})$, we get $P_3^1 = P_3^2 - 1$. Similarly, once the effect of O_2 is excluded in S^1 , we have $P_1^1 = P_1^2 - 1$. Hence P_3^1 and then $ET(O_3^{S^1}, O_1^{S^0})$ must be correct.

(3) $T(O_1) = \text{DEL}$ and $T(O_3) = \text{INS}$. Due to the given precondition, we have $P_2^2 < P_3^2 \leq P_1^2$. After excluding the effect of O_2 , regardless of its operation

type, we still have $P_3^1 \leq P_1^1$. First consider the case of $P_3^1 < P_1^1$, it is obvious that the assertion is true. Then consider the case of $P_3^1 = P_1^1$. According to our tie-breaking rule, when an insert is executed after a causally preceding delete at the same position, or when an insert is executed after a concurrent delete operation at the same position, the insert position is assumed to precede the delete position. That is, relation $C(O_3) \prec C(O_1)$ maintains. Hence $ET(O_3^{S^1}, O_1^{S^0})$ is correct.

(4) $T(O_1) = \text{INS}$. Due to the given precondition, we have $P_2^2 \leq P_3^2 \leq P_1^2$. After excluding the effect of O_2 , we have $P_3^1 \leq P_1^1$. Similar to the analysis of case (3), when $P_3^1 = P_1^1$, if two (concurrent or causally related) inserts occur at the same position, the contextually following one inserts to the left of contextually preceding one. That is, relation $C(O_3) \prec C(O_1)$ maintains and $ET(O_3^{S^1}, O_1^{S^0})$ must be correct.

The above four conditions cover all position operation type combinations for case $O_2 \prec O_3 \prec O_1$. The other two cases $O_2 \prec O_1 \prec O_3$ and $O_3 \prec O_2 \prec O_1$ can be proved similarly. Hence $ET(O_3^{S^1}, O_1^{S^0})$ is correct if $ET(O_3^{S^2}, O_2^{S^1})$ is correct. $\square$

Theorem 2. *Given an operation O^{S^n} and a negative order operation sequence, SQ_{ne} , such that $S^0 + SQ_{ne} = S^n$. When O is exclusively transformed against any element in SQ_{ne} on which it does not depend, $ET(O^{S^i}, SQ_{ne}[i])$ is always correct.*

Proof by induction: For convenience, we assume SQ_{ne} has n elements: $O_1^{S^0}, \dots, O_i^{S^{i-1}}, O_{i+1}^{S^i}, \dots, O_n^{S^{n-1}}$, and any operation $SQ_{ne}[i] \rightarrow O$.

$n = 1$: There is only one element $O_1^{S^0}$ in SQ_{ne} . When processing $ET(O^{S^1}, O_1^{S^0})$, S^1 is the generation state of O and $S^0 + O_1^{S^0} = S^1$. Due to our analysis in Section 6, the result of $ET(O^{S^n}, SQ_{ne}[n])$ is correct.

$n > 1$: Due to our analysis in Section 7, we exclusively transform O against SQ_{ne} from right to left. Assume the k th step $ET(O^{S^{n-k}}, SQ_{ne}[n-k])$ is correct. Now consider the $(k+1)$ th step. Due to our definition of negative order sequence, the three operations in question, $SQ_{ne}[n-(k+1)]$, $SQ_{ne}[n-k]$, and $O^{S^{n-k}}$, satisfy the preconditions in the above lemma, especially $SQ_{ne}[n-k] \prec SQ_{ne}[n-(k+1)]$. As a corollary, we can correctly process $ET(O^{S^{n-(k+1)}}, SQ_{ne}[n-(k+1)])$. $\square$

10.2. CORRECTNESS OF COMPUTING β AND δ

Theorem 3. *Given a contextually serialized sequence $SQ = [O_1^{S^0}, \dots, O_n^{S^{n-1}}]$, $S^0 + SQ = S^n$, and O^{S^n} is executed on S^n . Our approach can always find the β and δ values of O^{S^n} relative to S^0 .*

Proof by induction $n = 0$: O^{S^0} is executed on S^0 and SD is empty. Trivially $\beta_{S^0}(O) = P(O^{S^0})$ and $\delta_{S^0}(O) = 0$.

$n \rightarrow n + 1$: Assume the assertion holds for SQ with $n \geq 0$ operations. Let SD_n be a SD built from SQ , SD_n has m operations, and SQ_{ne} be a negative order sequence built from SD_n . Let $SQ_{ne} = [O_{y_1}^{S^0}, \dots, O_{y_k}^{S^{(k-1)'}}], \dots, O_{y_m}^{S^{(m-1)'}}]$. By definition of SD , there exists at most one insert operation $O_{y_k} \in SD_n$ which O depends on, i.e., $\beta_{S^0}(O_{y_k}) = \beta_{S^0}(O)$. Suppose there exist p operations in SD_n whose β values are less than $\beta_{S^0}(O)$ and q operations whose β values are greater than $\beta_{S^0}(O)$. Obviously $p + q \leq m$. We consider the following two cases:

$p + q = m$: In this case, O will not depend on any operation in SD_n . the result of ET between O and any operation in SQ_{ne} is always correct, based on the theorem. $\beta_{S^0}(O)$ is correct and $\delta_{S^0}(O)$ is always 0.

$p + q < m$: Let $O_{y_k}^{S^{(k-1)'}} = \text{insert}(S(O_{y_k}), P(O_{y_k}^{S^{(k-1)'}}))$. Then O will either insert a character into $S(O_{y_k})$ or delete a character from $S(O_{y_k})$. In other words, O depends on O_{y_k} and $\text{ET}(O^{S^k}, O_{y_k}^{S^{(k-1)'}})$ is not available. However, before O encounters $O_{y_k}^{S^{(k-1)'}}$ in the transformation path, O must have already been exclusively transformed against $[O_{y_{k+1}}^{S^{k'}}, \dots, O_{y_m}^{S^{(m-1)'}}]$. Due to the theorem, O must have correctly excluded the effects of these operations. Now we do not really need to process $\text{ET}(O^{S^{k'}}, O_{y_k}^{S^{(k-1)'}})$. If O is to insert a character into $S(O_{y_k})$, we have $\beta_{S^0}(O) = \beta_{S^0}(O_{y_k})$ and let $\delta_{S^0}(O)$ be the insert position in $S(O_{y_k})$. Similarly we can correctly determine $\beta_{S^0}(O)$ and $\delta_{S^0}(O)$ if O is to delete a character from $S(O_{y_k})$.

Therefore we can always correctly update SD_n to SD_{n+1} with a $(n + 1)$ th operation by its β and δ values that are correctly computed as above. Then we are also able to correctly build a negative order sequence from SD_{n+1} , based on which the β and δ values of another operation can be correctly computed. $\square$

10.3. SATISFACTION OF TP1 AND TP2

It has been shown in Ressel et al. (1996) that TP1 and TP2 are the necessary and sufficient conditions for ensuring convergence along arbitrary transformation paths. Hence we only need to prove that our IT functions can satisfy TP1 and TP2.

Proof of TP1: TP1 means different execution paths of any two concurrent operations O_1 and O_2 yield the same result. This must hold for every operation type combination (INS vs. INS, INS vs. DEL, DEL vs. INS, or DEL vs. DEL). However, since the procedure of proving any one of these four combinations is very similar, here we only elaborate the INS vs. INS case as follows.

Consider two concurrent operations based on the same state S^i : $O_1^{S^i} = \text{insert}(C(O_1), P(O_1^{S^i}))$ and $O_2^{S^i} = \text{insert}(C(O_2), P(O_2^{S^i}))$. Regardless of processing

$IT(O_1^{S^i}, O_2^{S^i})$ or $IT(O_2^{S^i}, O_1^{S^i})$, the position relation between $C(O_1)$ and $C(O_2)$ is unique due to the unambiguity of computing β (as proved by the above theorem) and the definition of effects relation between concurrent operations (Section 5). Without loss of generality, we assume $C(O_1) \prec C(O_2)$.

Now consider two different execution paths, $SQ_1 = [O_1^{S^i}, O_2^{S^{i+1}}]$ and $SQ_2 = [O_2^{S^i}, O_1^{S^{(i+1)'}}]$, where $S^i + SQ_1 = S^{i+2}$ and $S^i + SQ_2 = S^{(i+2)'}$. Note the position parameters of $O_1^{S^i}$ and $O_2^{S^i}$ are both defined (generated or executed) on state S^i before they are inclusively transformed against each other. For convenience, let $p_1 = P(O_1^{S^i})$, $p_2 = P(O_2^{S^i})$, $C_1 = C(O_1)$ and $C_2 = C(O_2)$. Suppose S^i looks like “... $C_{p_1-1}C_{p_1} \dots C_{p_2-1}C_{p_2} \dots$ ”.

In the first path (SQ_1), after O_1 is executed, state S^{i+1} becomes “... $C_{p_1-1}C_1C_{p_1} \dots C_{p_2-1}C_{p_2} \dots$ ”. Due to the insertion of character C_1 , which takes position p_1 , the original characters $C_{p_1}, \dots, C_{p_2-1}, C_{p_2} \dots$ are actually “pushed” to positions $p_1 + 1, \dots, p_2, p_2 + 1, \dots$, respectively. Then $O_2^{S^i}$ cannot be executed as it is in state S^{i+1} . Otherwise $C(O_2)$ would be inserted before C_{p_2-1} instead of before C_{p_2} . Hence the position parameter of O_2 needs to be shifted due to $O_1 \prec O_2$ and our IT functions. Specifically, $O_2^{S^{i+1}} = IT(O_2^{S^i}, O_1^{S^i}) = \text{insert}(C(O_2), P(O_2^{S^i}) + 1)$. Shifting of the execution position of O_2 incorporates the effect of O_1 because O_1 inserts one character to the left of the intended insert position of O_2 . This way $C(O_2)$ is correctly inserted, after which the state becomes $S^{i+2} = \dots C_{p_1-1}C_1C_{p_1} \dots C_{p_2-1}C_2C_{p_2} \dots$.

In the second path (SQ_2), $O_2^{S^i}$ is executed first. The state becomes $S^{(i+1)'} = \dots C_{p_1-1}C_{p_1} \dots C_{p_2-1}C_2C_{p_2} \dots$. Due to $O_1 \prec O_2$ and our IT functions, O_1 should be executed without shifting its position parameter, because the execution of O_2 does not affect the insert position of O_1 . That is, $O_1^{S^{(i+1)'}} = IT(O_1^{S^i}, O_2^{S^i}) = O_1^{S^i}$. This way C_1 is correctly inserted, after which the state becomes $S^{(i+2)'} = \dots C_{p_1-1}C_1C_{p_1} \dots C_{p_2-1}C_2C_{p_2} \dots$. As a result, $S^{i+2} = S^{(i+2)'}$. $\square$

Proof of TP2: The TP2 condition can be rephrased as follows. Given three concurrent operations, $O_1^{S^i}$, $O_2^{S^i}$, and $O_3^{S^i}$, when $O_1^{S^i}$ is inclusively transformed along different paths, $[O_3^{S^i}, O_2^{S^{(i+1)3}}]$ and $[O_2^{S^i}, O_3^{S^{(i+1)2}}]$, it will yield the same result, i.e., $IT(O_1^{S^{(i+1)2}}, O_3^{S^{(i+1)2}}) = IT(O_1^{S^{(i+1)3}}, O_2^{S^{(i+1)3}})$, where $S^i + [O_2^{S^i}] = S^{(i+1)2}$, $S^i + [O_3^{S^i}] = S^{(i+1)3}$, $O_1^{S^{(i+1)2}} = IT(O_1^{S^i}, O_2^{S^i})$, $O_1^{S^{(i+1)3}} = IT(O_1^{S^i}, O_3^{S^i})$, $O_2^{S^{(i+1)3}} = IT(O_2^{S^i}, O_3^{S^i})$, and $O_3^{S^{(i+1)2}} = IT(O_3^{S^i}, O_2^{S^i})$.

Similar to TP1, TP2 can be further rephrased as follows. Two different execution paths, $SQ_1 = [O_2^{S^i}, O_3^{S^{(i+1)2}}, O_1^{S^{(i+1)23}}]$ and $SQ_2 = [O_3^{S^i}, O_2^{S^{(i+1)3}}, O_1^{S^{(i+1)32}}]$, will yield the same result, i.e., $S^i + SQ_1 = S^i + SQ_2$, where $S^i + [O_2^{S^i}, O_3^{S^{(i+1)2}}] = S^{(i+1)23}$ and $S^i + [O_3^{S^i}, O_2^{S^{(i+1)3}}] = S^{(i+1)32}$.

Inspired from the proof of TP1, we know that if our IT functions are able to maintain the effects relation between these three concurrent operations regardless of the transformation paths, then the final result must be the same. In other words, the TP2 condition is satisfied if any two operations, say O_1 and O_2 , can maintain their effects relation despite that they

are respectively transformed against a third operation say O_3 . This is intuitive, because by our definitions, the effects relation between any two concurrent operations is only determined by their β values relative to their LSP, their position parameters, and their generation site IDs. If the effect relations always maintain, then the characters can always be inserted or deleted at the right positions, and thus the document states from different execution paths will always converge. Hence we define the following two conditions and proposition:

CC1: The effects relation between $O_1^{S^{(i+1)}_3}$ and $O_2^{S^{(i+1)}_3}$ is equivalent to the effects relation between $O_1^{S^i}$ and $O_2^{S^i}$.

CC2: The effects relation between $O_1^{S^{(i+1)}_2}$ and $O_3^{S^{(i+1)}_2}$ is equivalent to the effects relation between $O_1^{S^i}$ and $O_3^{S^i}$.

Proposition 1. *CC1 and CC2 are a sufficient condition of TP2. That is, if our IT functions can satisfy CC1 and CC2, then TP2 is satisfied.* $\square$

Due to the symmetry of CC1 and CC2, we only need to prove one of them. Here we choose to prove CC1. The following four lemmas cover all the possible cases. Note $\beta_{12}(O_1)$ and $\beta_{12}(O_2)$ are the beta values of O_1 and O_2 in their LSP, respectively.

Lemma 2. *CC1 holds when $\beta_{12}(O_1) \neq \beta_{12}(O_2)$.*

Proof: Due to the correctness of our approach to computing β , we always can correctly compute the values of $\beta_{12}(O_1)$ and $\beta_{12}(O_2)$ relative to the LSP of O_1 and O_2 . Due to Section 5, the LSP of O_1 and O_2 only depends on their vector timestamps, which will never change after the operations are generated. Hence, no matter from which state, S^i or $S^{(i+1)}_3$, β_{12} values are computed, the relation between $\beta_{12}(O_1)$ and $\beta_{12}(O_2)$ is the same and can uniquely determine the effects relation between O_1 and O_2 . Therefore the effects relation between $O_1^{S^{(i+1)}_3}$ and $O_2^{S^{(i+1)}_3}$ is equal to that between $O_1^{S^i}$ and $O_2^{S^i}$. $\square$

Lemma 3. *CC1 holds when $\beta_{12}(O_1) = \beta_{12}(O_2)$ and $T(O_3) = INS$.*

Proof: Without loss of generality, we assume $O_1^{S^i} \prec O_2^{S^i}$. There are three possible effects relations between $O_1^{S^i}$, $O_2^{S^i}$ and $O_3^{S^i}$ as follows: (1) $O_3^{S^i} \prec O_1^{S^i} \prec O_2^{S^i}$, (2) $O_1^{S^i} \prec O_2^{S^i} \prec O_3^{S^i}$, and (3) $O_1^{S^i} \prec O_3^{S^i} \prec O_2^{S^i}$. Accordingly, due to our IT functions, after processing $O_1^{S^{(i+1)}_3} = IT(O_1^{S^i}, O_3^{S^i})$ and $O_2^{S^{(i+1)}_3} = IT(O_2^{S^i}, O_3^{S^i})$, there are three possible results of $P(O_1^{S^{(i+1)}_3})$ and $P(O_2^{S^{(i+1)}_3})$:

- (1) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i}) + 1$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i}) + 1$,
- (2) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i})$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i})$,

(3) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i})$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i}) + 1$.

For any one of these three results, it is obvious that, due to our IT definition, $O_1^{S^{(i+1)}_3} \prec O_2^{S^{(i+1)}_3}$. $\square$

Lemma 4. *CC1 holds when $\beta_{12}(O_1) = \beta_{12}(O_2)$, $T(O_3) = \text{DEL}$, and at least one of O_1 and O_2 is a deletion.*

Proof. Following the same assumption of Lemma 3, there are three possible results of $P(O_1^{S^{(i+1)}_3})$ and $P(O_2^{S^{(i+1)}_3})$ correspondingly:

(1) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i}) - 1$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i}) - 1$,

(2) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i})$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i})$,

(3) $P(O_1^{S^{(i+1)}_3}) = P(O_1^{S^i})$ and $P(O_2^{S^{(i+1)}_3}) = P(O_2^{S^i}) - 1$.

For (1) and (2), it is obvious that $O_1^{S^{(i+1)}_3} \prec O_2^{S^{(i+1)}_3}$. For (3), if $T(O_1) = \text{DEL}$, we must have $P(O_1^{S^i}) + 1 < P(O_2^{S^i})$ because $O_1^{S^i} \prec O_3^{S^i} \prec O_2^{S^i}$ and $T(O_3) = \text{DEL}$. It is also obvious that the assertion $O_1^{S^{(i+1)}_3} \prec O_2^{S^{(i+1)}_3}$ is true. If $T(O_1) = \text{INS}$ and $T(O_2) = \text{DEL}$, similarly we must have $P(O_1^{S^i}) + 1 \leq P(O_2^{S^i})$. After inclusively transforming O_1 and O_2 against O_3 , we have $P(O_1^{S^{(i+1)}_3}) \leq P(O_2^{S^{(i+1)}_3})$. Hence $O_1^{S^{(i+1)}_3} \prec O_2^{S^{(i+1)}_3}$ due to our tie-breaking rule between an insertion and a deletion at the same position. $\square$

Lemma 5. *CC1 holds when $T(O_3) = \text{DEL}$ and $T(O_1) = T(O_2) = \text{INS}$.*

Proof. Following the assumption of Lemma 4, we also have the three possible effect relations between the three operations. For cases (1) and (2), it is easy to see that the assertion holds. For case (3) $O_1^{S^i} \prec O_3^{S^i} \prec O_2^{S^i}$, our proof follows.

Because $T(O_3) = \text{DEL}$, there must exist an inverse operation of O_3 , denoted as $\overline{O_3}$, which inserts character $C(O_3)$. Obviously $\overline{O_3} \rightarrow O_3$. To simplify discussion, we assume (a) O_3 is the first deletion that O_1 and O_2 have inclusively transformed against, and (b) $\overline{O_3}$ and O_3 are contextually serialized operations, meaning there is no other operations between them. The reason we can make these two assumptions is that (1) due to Lemma 3, the existence of insertions between $\overline{O_3}$ and O_3 does not affect the effects relation between O_1 and O_2 , and (2) if there exists another delete operation say O_4 before O_3 , we can just choose to consider O_4 instead O_3 , because it is a delete operation that is important here. Under these assumptions, we consider the following two cases:

First, $\overline{O_3} \rightarrow (O_1 \parallel O_2)$. Obviously O_1 and O_2 do not need to inclusively transform against $\overline{O_3}$. Meanwhile it means that the LSP between O_1 and O_2 must include character $C(\overline{O_3})$, and $C(\overline{O_3})$ must be between $C(O_1)$ and $C(O_2)$ relative to the LSP. Hence we have $\beta_{12}(O_1) < \beta_{12}(O_2)$. Because β values only depend on LSP, we must have $O_1^{S^{(i+1)}_3} \prec O_2^{S^{(i+1)}_3}$.

Second, $\overline{O_3} \parallel (O_1 \parallel O_2)$. If $\beta_{12}(O_1)\beta_{12}(O_1) < \beta_{12}(O_2)$, it is obvious the assertion holds. Now consider the case of $\beta_{12}(O_1) = \beta_{12}(O_2)$. When O_1 and O_2 are inclusively transformed against $\overline{O_3}$, the state $\overline{O_3}$ based on must be S^{i-1} , because $O_3^{S^i}$ immediately follows $\overline{O_3}$ due to our assumption $S^{i-1} + \overline{O_3}^{S^{i-1}} = S^i$. Due to $O_1^{S^i} \prec O_3^{S^i} \prec O_2^{S^i}$, there only exists one possible position relation: $P(O_1^{S^i}) + 1 \leq P(O_2^{S^i})$.

First consider the case where $P(O_1^{S^i}) + 1 < P(O_2^{S^i})$. After processing $IT(O_1^{S^i}, O_3^{S^i})$ and $IT(O_2^{S^i}, O_3^{S^i})$, we have $P(O_1^{S^{(i+1)3}}) < P(O_2^{S^{(i+1)3}})$. Thus, due to our IT functions, $O_1^{S^{(i+1)3}} \prec O_2^{S^{(i+1)3}}$.

Then consider the case where $P(O_1^{S^i}) + 1 = P(O_2^{S^i})$. According to Lemma 3, we know that $\overline{O_3}$ as an insert operation does not change the position relation between O_1 and O_2 . Hence consistently to our assumption $O_1^{S^i} 2^{S^i}$ we have $O_1^{S^{i-1}} \prec O_2^{S^{i-1}}$. After excluding the effect of $\overline{O_3}$, we have $P(O_1^{S^{i-1}}) = P(O_2^{S^{i-1}})$. Due to our IT functions, it must be $ID(O_1) < ID(O_2)$. In state $S^{(i+1)3}$, again without character $C(\overline{O_3})$, it is obvious that $P(O_1^{S^{(i+1)3}}) = P(O_2^{S^{(i+1)3}})$. According to $ID(O_1) < ID(O_2)$ and our IT functions, we can conclude $O_1^{S^{(i+1)3}} 2^{S^{(i+1)3}}$. $\square$

The above four lemmas cover all possible cases of CC1. Therefore CC1 holds and similarly CC2 can be proved. That is, our IT functions satisfy the TP2 condition.

Theorem 4. *The SDT algorithm ensures convergence in the presence of arbitrary transformation paths.*

The above proofs have shown that our IT functions satisfy TP1 and TP2. It has been proved in Ressel et al. (1996) that TP1 and TP2 are the necessary and sufficient conditions of IT for an OT-based algorithm to ensure convergence along arbitrary transformation paths. $\square$

The proof of Ressel et al. (1996) is based on a N-dimension interaction model. Any OT algorithm can ensure convergence in the presence of arbitrary transformation paths, if (1) its IT functions satisfy TP1 and TP2, and (2) its integration procedure effectively implements the interaction model. Our algorithm (including its IT functions and integration procedure) is the only approach to our knowledge that satisfies both these two conditions. A number of other algorithms, such as adOPTed (Ressel et al., 1996) and GOTO (Sun et al., 1998; Sun and Ellis, 1998; Sun, 2002b), satisfy condition (2) but their IT functions do not satisfy TP2. Hence they cannot ensure convergence without extra measures such as Sun et al. (1998), Shen and Sun (2002), Vidot et al. (2000) in the integration procedure.

10.4. SATISFACTION OF THE CSM CONSISTENCY MODEL

As defined in Section 4, we say a group editor is consistent iff it can preserve causality, single-operation effects, and multi-operation effects relation. Due

to our integration procedure (Section 8), we use vector timestamps and require that only causally ready operations can be integrated. Hence our algorithm guarantees causality preservation. Due to the above proofs, our IT functions can correctly *maintain* the effects relation between concurrent operations along arbitrary transformation paths. Now we prove that our algorithm can maintain the correct effects relation between concurrent operations.

In the following, we first explicitly define Proposition 2, which has been an implicit assumption of all group editors. Then Theorem 5 asserts that the algorithm achieves single-operation effects preservation (S) and multi-operation effects relation preservation (M) given that the IT functions are correct. $IT(O_1, O_2)$ is correct means that, for any two concurrent operations O_1 and O_2 , it can correctly determine their effects relation and include the effect of O_2 by adjusting the position parameter of O_1 . After that, we will prove that our IT functions are correct, by first proving a number of lemmas. Then together with Theorem 4 we can assert that our IT functions can maintain the correct effects relation between any concurrent operations and that our algorithm can achieve CSM consistency.

Proposition 2. *For any operation O , let S_i^p be its generation state. $P(O^{S_j^u})$ must be the correct execution position of O relative to S_j^u , if $S_j^u = S_i^p$. The execution of O in state S_i^p or S_j^u will not change the position relation of existing characters in its execution state.*

Theorem 5. *Assuming that our IT functions are correct, the SDT algorithm can achieve single-operation effects preservation and multi-operation effects relation preservation.*

Proof. Without possible interferences from concurrent operations, when an operation O is generated at one site and executed later at any other sites, its execution state is always equal to its generation state. Due to the above proposition, achieving the “S” condition is obvious. In absence of concurrent execution, the “M” condition is not required (or trivially satisfied). In fact, OT is not necessary when there is no concurrent execution at all.

Considering the effects of concurrent operations, the execution states of an operation could be different from its generation state. Given any operation O that is generated at site i and its generation state is S_i^m . Without loss of generality, assume all sites start from the same empty initial state S^0 . We consider how O is synchronized at site j where $i \neq j$. Assume that when O is ready to be synchronized, a sequence SQ of n operations ($n \geq 1$) have been executed in S_j^0 and the execution state of O is now S_j^n . To correctly execute O in state S_j^n , we need to achieve S and M . That is, in state S_j^f that is resulted

from executing O in S_j^n , the correct position relation between $C(O)$ and characters belonging to S_i^p must be maintained, and the correct position relation between $C(O)$ and characters concurrently inserted at site j should also be maintained. Suppose in sequence SQ there are p operations that causally precede O and q operations that are concurrent with O . Thus $p + q = n$. Consider the following three cases.

First, if $p = n$ or all operations in SQ are causally preceding O , since O is causally ready, we can say that $S_j^n = S_i^m$, that is, the execution state of O is equivalent to its generation state. Due to Proposition 2, executing O as it is in state S_j^n will satisfy both S and M conditions.

Second, consider the case where $q = n$, namely, all operations in SQ are concurrent with O . Because the initial state S^0 is empty, executing O anywhere trivially satisfies the S condition. Due to our assumption that IT can correctly determine the effects relation between all operations in SQ and O , we are able to correctly adjust the position parameter of O to include the effects of these executed concurrent operations such that the result can be correctly executed in state S^n . Hence the M condition is also satisfied.

Third, suppose SQ is a mixed sequence of operations among which some causally precede O and some are concurrent with O . That is, $p + q = n$ and $0 < p, q < n$. Due to Theorem 4, we can find another transformation path SQ' from S_j^0 to S_j^n in which the p operations that causally precede O are executed before those q operations that are concurrent with O . Let S^p be the state resulted from executing the p causally preceding operations in S^0 . Due to Proposition 2, the original position parameter of O is correct relative to state S^p . In other words, executing O as it is in state S^p satisfies the S condition. Then due to our assumption that IT can correctly determine the effects relation between those q concurrent operations, we can correctly adjust the position parameter of O^{S^p} to incorporate the effects of those q operations such that the resulted form $O^{S^{p+q}}$ or O^{S^n} can be correctly executed in state S^n . Thus the M condition is also satisfied. $\square$

Lemma 6. *For any two concurrent operations, $O_1 \parallel O_2$, if $IT(O_1^{S^p}, O_2^{S^p})$ is correct in a given state S^p , then $IT(O_1^{S^q}, O_2^{S^q})$ and $IT(O_2^{S^q}, O_1^{S^q})$ are both correct for any possible state S^q in the same interaction model.*

Proof. Let $O_1^{S^{p+1}} = IT(O_1^{S^p}, O_2^{S^p})$, $O_1^{S^{q+1}} = IT(O_1^{S^q}, O_2^{S^q})$, $S^{p+1} = S^p + O_2^{S^p}$, and $S^{q+1} = S^q + O_2^{S^q}$. If the effects relation between O_1 and O_2 as determined in $IT(O_1^{S^q}, O_2^{S^q})$ is different from that determined in $IT(O_1^{S^p}, O_2^{S^p})$, the relative position between $C(O_1)$ and $C(O_2)$ in state S^{q+1} will be different from that in state S^{p+1} . Due to causality preservation, all operations executed after O_1 must happen after or be concurrent with O_1 . Due to Proposition 2, their execution will not change the position relations between $C(O_1)$ and $C(O_2)$ in either state S^{p+1} or S^{q+1} . As a result, the execution path via S^p will yield

divergent state from the path via S^q , which contradicts the Theorem 4. Therefore if $IT(O_1^{S^p}, O_2^{S^p})$ is correct, then $IT(O_1^{S^q}, O_2^{S^q})$ must be correct. Due to TP1, $IT(O_2^{S^q}, O_1^{S^q})$ must also be correct. $\square$

Lemma 7. *Given any operation O that is generated in an empty initial state S^0 . Suppose there exists a sequence SQ with n operations such that $S^0 + SQ = S^n$ and O does not appear in SQ . All operations in SQ are causally related and each of them is concurrent with O , namely, $SQ[i] \rightarrow SQ[i+1]$ and $SQ[i] \parallel O$, for any $i=1, \dots, n-1$. Then $IT(O, SQ[i])$ is correct.*

Proof. Because S^0 is empty and a deletion cannot be processed in an empty state, O must be an insertion. The LSP of O and any $SQ[i]$ is S^0 and $\beta_{lsp}(O) = \beta_{lsp}(SQ[i]) = 0$. We prove this assertion by induction. When $n=1$, because $SQ[1]$ is generated in S^0 , it must also be an insertion and $P(O) = P(SQ[1]) = 0$. Since there is no inherent effects relation between O and $SQ[1]$, we impose an order on them by comparing their site ids. Due to our defined IT functions, $IT(O, SQ[1])$ can correctly determine the effects relation between them in this case.

Assume that when $n=k$, $IT(O, SQ[i])$ is correct for any $i \leq k$. Then $P(O^{S^k})$ must be the correct execution position of O relative to S^k , where $S^k = S^0 + SQ[1, k]$. Consider the case where $n = k+1$. Since S^k is the generation state of $SQ[k+1]$, due to Proposition 2, $P(SQ[k+1])$ must be the correct relative to S^k . In other words, the position relations between $C(O)$ and other characters in S^k are correct. So are the relations between $C(SQ[k+1])$ and those characters in S^k .

There are three possible relations between $P(O^{S^k})$ and $P(SQ[k+1])$: (1) $P(O^{S^k}) < P(SQ[k+1])$, (2) $P(O^{S^k}) = P(SQ[k+1])$, and (3) $P(O^{S^k}) > P(SQ[k+1])$. Note cases (1) and (3) are symmetric. We only need to consider (1). It means that there must be at least one landmark character between the execution position of O^{S^k} and $SQ[k+1]$ relative to S^k . Thus $O \prec SQ[k+1]$. Assume C_j is the landmark and $SQ[j]$ is the operation that inserts C_j . Obviously $j \leq k$ and $O \prec SQ[j] \prec SQ[k+1]$.

Now we prove that such a relation among $O, SQ[j]$ and $SQ[k+1]$ is not introduced due to the transformation of O against some operation in SQ . We prove this by contradiction. Since relation $SQ[j] \prec SQ[k+1]$ is correct, we assume that $SQ[j] \prec O$. However, due to the induction assumption, $IT(O, SQ[i])$ is correct for any $i \leq k$. Hence the effects relation of $SQ[j]$ and O is correct and it is maintained in state S^k . Due to $SQ[j] \prec O$ and $SQ[j] \prec SQ[k+1]$, then C_j is not a landmark between the execution position of O and $SQ[k+1]$ relative to S^k , which is absurd.

Consider case (2), $P(O^{S^k}) = P(SQ[k+1])$. There is no character between them. We still use proof by contradiction. Assume that there is one character C_j between $P(O^{S^k})$ and $P(SQ[k+1])$, and $SQ[j]$ is the operation that inserts

C_j . Obviously $j \leq k$. Without loss of generality, assume that $O^{S^k} \prec SQ[j] \prec SQ[k+1]$. Since S^k is the generation state of $SQ[k+1]$ which already includes C_j , it must be that $P(SQ[j]) < P(SQ[k+1])$. Besides, due to the correctness of $IT(O, SQ[j])$ and $O \prec SQ[j]$ and that both O and $SQ[j]$ are insertion, it must be $P(O) \leq P(SQ[j])$. Hence $P(O) \leq P(SQ[j]) < P(SQ[k+1])$, which contradicts $P(O^{S^k}) = P(SQ[k+1])$.

Therefore our defined IT functions can correctly determine the effects relation between O and $SQ[k+1]$ and thus any operation in SQ . $\square$

Lemma 8. *Given two causally related sequences SQ_1 with n operations and SQ_2 with m operations that are generated from the same empty initial state S^0 at different sites, respectively. No operation appears in both SQ_1 and SQ_2 . Then $IT(SQ_1[i], SQ_2[j])$ can correctly determine the relation between any $SQ_1[i]$ and any $SQ_2[j]$.*

Proof. From the preconditions, obviously we have $SQ_1[i] \parallel SQ_2[j]$, their LSP is the empty initial state S^0 , and $\beta_{lsp}(SQ_1[i]) = \beta_{lsp}(SQ_2[j]) = 0$. When $i=1$, due to Lemma 7, $IT(SQ_1[1], SQ_2[j])$ is correct for any $j \in [1, m]$.

Assume that when $1 \leq i \leq k-1$, $IT(SQ_1[i], SQ_2[j])$ is correct, where $j \in [1, m]$. Suppose that after $SQ_1[1, k-1]$ and $SQ_2[1, m]$ are executed on S^0 , the final state becomes S^f . Consider two different transformation paths from S^0 to S^f : path $p_1 = SQ_2 + SQ'_1[1, k-1]$ and path $p_2 = SQ_1[1, k-1] + SQ'_2$.

Consider the case where $i=k$. Due to Theorem 4 and Lemma 6, the problem can be switched from p_1 to p_2 by proving that $IT(SQ_1[k], SQ'_2[j])$ is correct for any $j \in [1, m]$. We prove it by induction on j .

When $j=1$: due to Lemma 7, $IT(SQ_1[k], SQ_2[1])$ is correct. Hence $IT(SQ_1[k], SQ'_2[1])$ is also correct.

Assume that $IT(SQ_1[k], SQ'_2[j])$ is correct for any $j \leq p-1$. Then examine the case of $j=p$. Let S'' be the state these two operations are based on. Relative to S'' , there are three possible relations between $P(SQ_1[k])$ and $P(SQ'_2[p])$: (1) $P(SQ_1[k]) < P(SQ'_2[p])$, (2) $P(SQ_1[k]) = P(SQ'_2[p])$, and (3) $P(SQ_1[k]) > P(SQ'_2[p])$.

For cases (1) and (3), suppose there is a character C_i between $P(SQ_1[k])$ and $P(SQ'_2[p])$ in state S'' . Let O_i be the operation that inserts C_i . Note that O_i could be in SQ_1 or SQ_2 . Without loss of generality, assume $O_i \in SQ_1[1, k-1]$. Then the effects relation between O_i and $SQ_1[k]$ must be correct in S'' due to causality and Proposition 2. The effects relation between O_i and $SQ'_2[p]$ must also be correct in S'' due to the (outer) induction assumption. Hence we can correctly determine the position relation between $P(SQ_1[k])$ and $P(SQ'_2[p])$.

For case (2), there is no character between $C(SQ_1[k])$ and $C(SQ'_2[p])$. We prove it by contradiction. Without loss of generality, assume there is a character C_i between them and $C(SQ_1[k]) \prec C_i \prec C(SQ'_2[p])$. If O_i be the

operation in $SQ_1[1, k-1]$ that inserts C_t . Due to causality, C_t must appear in the generation state of $SQ_1[k]$. Because O_t is concurrent with all operations in SQ'_2 , no operation in SQ'_2 can delete C_t . Thus C_t must appear in S^u . Similarly if $O_t \in SQ_2[1, p-1]$, C_t must also appear in S^u . Due to the induction assumptions, the position relation between C_t and $C(SQ_1[k])$ is correct and so is the position relation between C_t and $C(SQ'_2[p])$ in S^u . Due to $C(SQ_1[k])_t \prec C(SQ'_2[p])$ and $T(O_t) = \text{INS}$, we get $P(SQ_1[k]) \prec P(SQ'_2[p])$, which contradicts the precondition $P(SQ_1[k]) = P(SQ'_2[p])$.

Therefore our IT functions can correctly determine the position and then the effects relation between $SQ_1[k]$ and $SQ'_2[p]$, and thus the effects relation between any $SQ_1[i]$ and any $SQ_2[j]$. $\square$

Lemma 9. *Given two operations, O_1 and O_2 , that are generated in states S^1 and S^2 , respectively, $T(O_1) = \text{INS}$ and $T(O_2) = \text{DEL}$. Let S^{lsp} be their LSP. Suppose $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$ and $\delta(O_2) = 0$. That is, O_2 deletes a character that is originally in S^{lsp} . Then $IT(O_1^{S^k}, O_2^{S^k})$ or $IT(O_2^{S^k}, O_1^{S^k})$ is correct for any state S^k where $S^{lsp} \leq S^1, S^2 \leq S^k$.*

Proof. We prove this lemma in the following five cases.

Case 1: $S^k = S^1 = S^2 = S^{lsp}$. Obviously $P(O_1^{S^k}) = P(O_2^{S^k})$. Based on our IT functions, the assertion is true.

Case 2: $S^2 = S^{lsp}$ and $S^1 > S^{lsp}$. Assume that there is an operation sequence SQ_1 , all members of which are causally related, and $S^{lsp} + SQ_1 = S^1$. We first prove by induction that $IT(O_2, SQ_1[i])$ is correct for any operation in SQ_1 .

$i = 1$: Based on the result of the case 1, $IT(O_2, SQ_1[1])$ is correct.

$n \rightarrow n+1$: Assume that $IT(O_2, SQ_1[i])$ is correct for any $i \leq n$. Then the relations between the effect characters of O_2 and $SQ_1[1, n]$ are correctly maintained in the generation state of $SQ_1[n+1]$, denoted as S_1^n . If $\beta_{lsp}(O_2) \neq \beta_{lsp}(SQ_1[n+1])$, then $IT(O_2, SQ_1[n+1])$ is obviously correct. If $\beta_{lsp}(O_2) = \beta_{lsp}(SQ_1[n+1])$ and $T(SQ_1[n+1]) = \text{INS}$, then $P(O_2^{S_1^n}) \geq P(SQ_1[n+1])$ and $IT(O_2, SQ_1[k+1])$ is correct. If $\beta_{lsp}(O_2) = \beta_{lsp}(SQ_1[n+1])$, $T(SQ_1[n+1]) = \text{DEL}$ and $\delta_{lsp}(SQ_1[n+1]) \neq 0$, then $P(O_2^{S_1^n}) > P(SQ_1[n+1])$ and $IT(O_2, SQ_1[n+1])$ is also correct.

Based on the above result, the relations between the effect characters of O_2 and any member of SQ_1 are correctly maintained in state S^1 . Due to $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$, $\delta(O_2) = 0$, $T(O_1) = \text{INS}$ and $T(O_2) = \text{DEL}$, we have $P(O_1^{S^1}) \leq P(O_2^{S^1})$. Hence $IT(O_2^{S^1}, O_1^{S^1})$ is correct. By Lemma 6, the assertion is true.

Case 3: $S^1 = S^{lsp}$ and $S^2 > S^{lsp}$. The assertion can be proved similarly to case 2.

Case 4: $S^1 > S^{lsp}$, $S^2 > S^{lsp}$ and $S^1 = S^k$. Let SQ_1 and SQ_2 be two causally related sequences such that $S^{lsp} + SQ_1 = S^1$ and $S^{lsp} + SQ_2 = S^2$. Assume that IT between any members of SQ_1 and SQ_2 do not satisfy the preconditions of this lemma. Due to Lemma 8, the inclusion transformation between any

members of SQ_1 and SQ_2 can be correctly processed. Similarly to the proof of Lemma 8, we chose a transformation path $SQ_2 + SQ'_1$, where SQ'_1 is the result of including the effects of SQ_2 into SQ_1 . Then we can similarly prove by induction that the transformation between O_2 and any member of SQ'_1 is correct. Finally by analyzing the relation between $P(O_1^{S^1})$ and $P(O_2^{S^1})$, the assertion is true.

Case 5: $S^1 > S^{lsp}$, $S^2 > S^{lsp}$ and $S^2 = S^k$. The proof is similar to case 4. $\square$

Lemma 10. *Given two operations, O_1 and O_2 , that are generated in states S^1 and S^2 , respectively, $T(O_1) = DEL$ and $T(O_2) = DEL$. Let S^{lsp} be their LSP. Suppose $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$, $\delta(O_1) \neq 0$ and $\delta(O_2) = 0$. Then $IT(O_1^{S^k}, O_2^{S^k})$ or $IT(O_2^{S^k}, O_1^{S^k})$ is correct for any state S^k such that $S^{lsp} \leq S^1, S^2 \leq S^k$.*

Similarly this lemma can be proved as Lemma 9. $\square$

Theorem 6. *Our IT functions are correct between any concurrent operations.*

Proof. This is to prove that $IT(O_1^S, O_2^S)$ can correctly determine the effects relation between any two concurrent operations O_1 and O_2 . We examine the following three cases: First, consider the case where there is at least one character between the effect positions of O_1 and O_2 relative to their LSP. That is, $\beta_{lsp}(O_1) \neq \beta_{lsp}(O_2)$. Due to the proved correctness of computing β , $IT(O_1^S, O_2^S)$ can correctly determine their effects relation by only comparing the β values.

Second, consider the case where at least one of O_1 and O_2 is a deletion which deletes a character that was originally in S^{lsp} , and $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$. Due to Lemmas 9 and 10, the assertion is true in this case. Note that we do not need to consider the case where both O_1 and O_2 are deletions, their effect characters appear in S^{lsp} , and $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$. The reason is that in such a case, they attempt to delete the same character and these two operations are considered as one operation.

Third, we consider the other three cases under the condition of $\beta_{lsp}(O_1) = \beta_{lsp}(O_2)$: (1) $T(O_1) = INS$, $T(O_2) = DEL$, and $\delta_{lsp}(O_2) \neq 0$; (2) $T(O_1) = DEL$ and $T(O_2) = DEL$, $\delta_{lsp}(O_1) \neq 0$, and $\delta_{lsp}(O_2) \neq 0$; (3) $T(O_1) = INS$ and $T(O_2) = INS$. In either case, the two concurrent operations insert or delete between the positions of two characters that were originally in S^{lsp} . Then the presence of characters in S^{lsp} does not make any difference. Hence we can assume that S^{lsp} is empty.

We reformulate the assertion as follows: Given two sites i and j start from the same empty initial state S^0 . Site i executes a sequence SQ_1 with n operations which results in state S_i^n . Site j executes a sequence SQ_2 with m operations which results in state S_j^m . Assume that $SQ_1[p] \parallel SQ_2[q]$ for any pair of operations in SQ_1 and SQ_2 , and no operation appear in both SQ_1

and SQ_2 . In addition, O_1 and O_2 are two concurrent operations, O_1 is executed (but may not be generated) in state S_i^n , and O_2 is executed (but may not be generated) in state S_j^m . We want to prove the correctness of $IT(O_1, O_2)$.

Since we have proved Lemma 8, here we only need to see if $IT(O_1, O_2)$ can be translated to satisfy the preconditions of Lemma 8. Take SQ_1 as an example and consider the following three cases:

- (1) All operations in SQ_1 causally precede O_1 .
- (2) All operations in SQ_1 are concurrent with O_1 . Due to Theorem 4, we can find an alternative path in which O'_1 (a transformed version of O_1) is directly executed in S_i^0 and SQ'_1 (SQ_1 transformed) follows.
- (3) p operations in SQ_1 are concurrent with O_1 and q operations in SQ_1 causally precede O_1 , where $0 \leq p, q \leq n$ and $p + q = n$. We can also find an alternative path where the q operations causally preceding O_1 are executed S_i^0 , after which O_1 is executed, and then the p operations concurrent to O_1 follow.

Hence SQ_1 can be transformed such that all operations executed before O_1 , if any, causally precede O'_1 . Similarly, SQ_2 can also be transformed such that all operations executed before O_2 , if any, causally precede O'_2 . This is exactly the precondition of Lemma 8. Due to Lemmas 8 and 6, $IT(O_1, O_2)$ can correctly determine the effects relation between O_1 and O_2 . $\square$

The proof of Theorem 6 may suggest it necessary in our IT functions (Algorithm 3) to compare δ in addition to β , P, and ID parameters. Here we prove that δ values are not really necessary. Note although the δ parameter is not used in our IT functions, it is useful elsewhere, e.g., in building SDs (Algorithm 8).

Theorem 7. *The δ_{lsp} values are not necessary for determining the effects relation $\prec$ between any two concurrent operations.*

Proof. Due to the above analysis, δ_{lsp} is used only when (1) the two concurrent operations have the same β_{lsp} value, (2) one (say O_1) is an insertion and the other (say O_2) is a deletion, and (3) $\delta_{lsp}(O_2) = 0$. It means that O_2 deletes the character at position β_{lsp} while O_1 inserts a new character at the same position. As a convention of group editors, we always have $O_1 \prec O_2$. Without loss of generality, assume we process IT between O_1 and O_2 relative to any possible state S^p . Then the execution positions of O_1 and O_2 relative to state S^p must be consistent with their effects relation $O_1 \prec O_2$. That is, $P(O_1^{S^p}) \not\prec P(O_2^{S^p})$, or $P(O_1^{S^p}) \leq P(O_2^{S^p})$. In other words, by comparing their position parameters relative to S^p , the state where IT between them is processed, we are already able to determine the effects relation between O_1 and O_2 . The rule can be formalized as follows: If $\beta_{lsp} O_1 = \beta_{lsp}(O_2)$,

$T(O_1) = \text{INS}$, $T(O_2) = \text{DEL}$, and $P(O_1^{S'}) \leq P(O_2^{S'})$, then $O_1 \prec O_2$. Obviously, it is not necessary to use their δ values. $\square$

11. Related work

Most previous research efforts on OT have been motivated by flaws in the seminal dOPT algorithm of Ellis and Gibbs (1989), specifically the dOPT puzzle and the TP2 puzzle. The dOPT puzzle has now been well solved in the literature, e.g., (Nichols et al., 1995; Ressel et al., 1996; Sun et al., 1998; Suleiman et al., 1998). The TP2 puzzle scenarios have been identified in Imine et al. (2003), Suleiman et al. (1998), Sun et al. (1998). The adOPTed algorithm (Ressel et al., 1996) cannot achieve convergence in the scenario shown in Figure 1, according to Sun et al. (1998). However, previous work seems to have only realized and addressed the divergence problem in those scenarios.

The TP2 puzzle has been approached from two different directions. One is to extend the original control algorithm of dOPT such that there is only one transformation path at all sites and thus TP2 is no longer a necessary condition of convergence, as in GOT (Sun et al., 1998), NICE (Shen and Sun, 2002), SOCT3/4 (Vidot et al., 2000), and TIBOT (Li et al., 2004). The second direction is to extend the original IT functions of dOPT such that they are able to satisfy the TP2 condition, as in SOCT2 (Suleiman et al., 1998) and IMOR (Imine et al., 2003).¹

NICE uses a notification server so that clients take turns to propagate operations through the server. SOCT3/4 uses a sequencer to maintain a global total order of execution. TIBOT defines rules that constrain operation propagation and synchronization. The presented solution to the convergence problem seems more general and applicable in peer-to-peer and mobile computing environments in which communication may be arbitrary or a central component may not be possible. GOT enforces a global total order of execution and is also able to achieve convergence in a fully replicated system. Nonetheless, NICE, SOCT3/4, and TIBOT cannot ensure operation effects relation. For example, at site 3 of Figure 1, NICE, SOCT3/4 and TIBOT will generate the same wrong result “a12c”. GOT works in Figure 1 but fails in other more complicated scenarios, as will be shown in Section 11.1.

Algorithms adOPTed (Ressel et al., 1996) and GOTO (Sun and Ellis, 1998; Sun, 2002b) assume that their IT functions satisfy TP2. They cannot achieve convergence without extra measures as in Shen and Sun (2002), Vidot et al. (2000). According to Sun (2002b), the problems with ET as discussed in Section 6 are avoided in GOTO because it only performs IT/ET between concurrent operations. According to Sun (2004), their solutions to the “false-tie” problem are implemented in CoWord (Sun et al., 2004) but not published. The solutions are not included in Sun et al. (1998), Shen and

Sun (2002), Sun and Ellis (1998), Sun (2002b). Our approach is independent and different.

11.1. GOT

GOT ensures convergence by enforcing a global total order of execution through an undo/do/redone scheme. Operations that have been executed out of order are undone first and then redone after the in-order operations are done. The total order " $\Rightarrow$ " in GOT is defined as follows: Given any two operations O_i and O_j , $O_i \Rightarrow O_j$, iff (1) $O_i \rightarrow O_j$, or (2) $\text{sum}(\text{V}(O_i)) < \text{sum}(\text{V}(O_j))$, or (3) $\text{ID}(O_i) < \text{ID}(O_j)$. Function *sum* is defined as a summation of all elements in the given vector timestamp. Although GOT does not cause problem in the TP2 scenario as shown in Figure 1, it fails in other more complicated scenarios such as the one shown in Figure 5.

Consider the counterexample in Figure 5. The initial document state is $S^0 = \text{"abc"}$ at all sites. User A continuously performs three operations at site 1: $O_1 = \text{insert("1",0)}$, $O_2 = \text{insert("2",1)}$, and $O_3 = \text{insert("x",4)}$. User B concurrently performs operation $O_4 = \text{delete("b",1)}$ at site 2. Meanwhile user C at site 3 executes two operation, $O_5 = \text{insert("3",3)}$ and $O_6 = \text{insert("y",1)}$. By definition, the total order of these operations is $O_1 \Rightarrow O_4 \Rightarrow O_5 \Rightarrow O_2 \Rightarrow O_6 \Rightarrow O_3$. When O_3 arrives at site 3, the operation log is $[O_1, O_4^{S_3^1}, O_5^{S_3^2}, O_2^{S_3^3}, O_6^{S_3^4}]$, where $S_3^1 = S_3^0 + [O_1]$, $S_3^2 = S_3^0 + [O_1, O_4^{S_3^1}]$, $S_3^3 = S_3^0 + [O_1, O_4^{S_3^1}, O_5^{S_3^2}]$, and $S_3^4 = S_3^0 + [O_1, O_4^{S_3^1}, O_5^{S_3^2}, O_2^{S_3^3}]$.

Due to the GOT control algorithm, we must first undo those operations that should go behind O_3 in the total order, then perform O_3 and redo those undone operations. However, because O_3 is the last operation in the total order, no operation actually needs to be undone. We only need to transform O_3 such that it can be executed in the current state of site 3. First, we restore

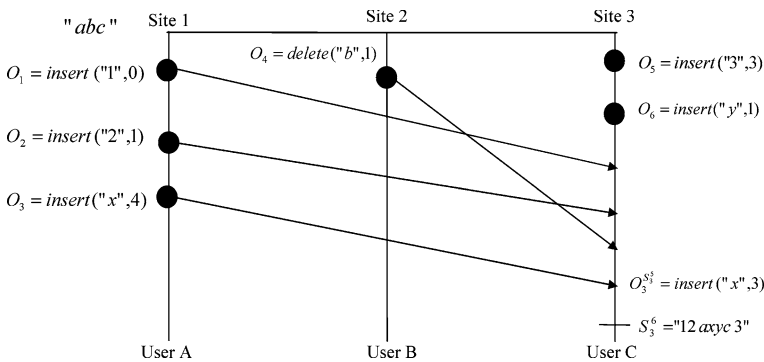


Figure 5. A puzzle scenario where GOT fails to preserve the correct character order.

the original form of O_2 by exclusively transforming $O_2^{S_3^3}$ against $O_5^{S_3^2}$ and $O_4^{S_3^1}$. After that, we exclusively transform O_3 against O_2 , which results in $O_3^{S_1} = \text{insert}(\text{"x"}, 3)$. Then we need to inclusively transform $O_3^{S_1}$ against sequence $[O_4^{S_1^1}, O_5^{S_2^2}, O_2^{S_3^3}, O_6^{S_3^4}]$. After O_3 has been inclusively transformed against $[O_4^{S_1^1}, O_5^{S_2^2}, O_2^{S_3^3}]$, we get $O_3^{S_3^4} = \text{insert}(\text{"x"}, 3)$. Now, $O_3^{S_3^4}$ is ready to be inclusively transformed against $O_6^{S_3^4}$, which is $\text{insert}(\text{"y"}, 3)$. Due to the IT functions of Sun et al. (1998), we have $O_3^{S_3^5} = \text{IT}(O_3^{S_3^4}, O_6^{S_3^4}) = \text{insert}(\text{"x"}, 3)$. After $O_3^{S_3^5}$ is executed, the state of site 3 becomes "12axyc3".

This is different from the correct result "12ayxc3". The reason follows. By executing O_1, O_2, O_3 in a row, the intention of user A is actually to insert 'x' after the landmark character 'b'. On the other hand, by executing O_5, O_6 concurrently, user C intends to insert 'y' before landmark 'b'. Hence in the correct order 'x' should go after 'y'. This problem is signified if the landmark character 'b' is recovered by selectively undoing operation O_4 .

11.2. IMOR

IMOR (Imine et al., 2003) uses one new parameter in the insert operation, as $\text{insert}(c, p, ip)$, where ip is the position of the operation relative to its generation state. When inclusively transforming two concurrent insertions whose positions tie, their ip values are compared, after which the ASCII codes of the characters are compared if there is still a tie. Intuitively, it may not make sense to compare the ip values of two concurrent operations if they are not defined on the same state. The nature of this problem resembles that of the dOPT puzzle (Sun and Ellis, 1998).

The counterexample in Figure 6 shows that IMOR is not able to solve the divergence problem. Suppose all the four sites in Figure 6 start with the same document state $S^0 = \text{"0123"}$. User A performs $O_1 = \text{insert}(\text{"b"}, 2)$ at site 1. Concurrently, user B performs $O_2 = \text{delete}(\text{"1"}, 1)$ at site 2, and the user D performs $O_4 = \text{insert}(\text{"a"}, 1)$ at site 4. After O_4 arrives at site 3 and is executed, user C performs $O_3 = \text{insert}(\text{"c"}, 2)$ at site 3. First consider site 4. Assume that O_2 arrives first. After processing $\text{IT}(O_2, O_4)$, $O_2^{S_4^1} = \text{delete}(\text{"1"}, 2)$ and $S_4^0 + [O_4] = S_4^1$. Similarly, when O_3 arrives, we get $O_3^{S_4^1} = \text{IT}(O_3, O_2^{S_4^1}) = \text{insert}(\text{"c"}, 2)$ and $S_4^1 + [O_2^{S_4^1}] = S_4^2$. Finally, O_1 is received. After inclusively transforming O_1 against $[O_4, O_2^{S_4^1}]$, we get $O_1^{S_4^2} = \text{insert}(\text{"b"}, 2)$. Then what will be the result of $\text{IT}(O_1^{S_4^2}, O_3^{S_4^1})$? Due to the IT functions defined in Imine et al. (2003), since $P(O_1^{S_4^2}) = P(O_3^{S_4^1})$ and their original execution positions ($\text{IP}(O_1) = 2$ and $\text{IP}(O_3) = 2$) are also the same, then we compare the character

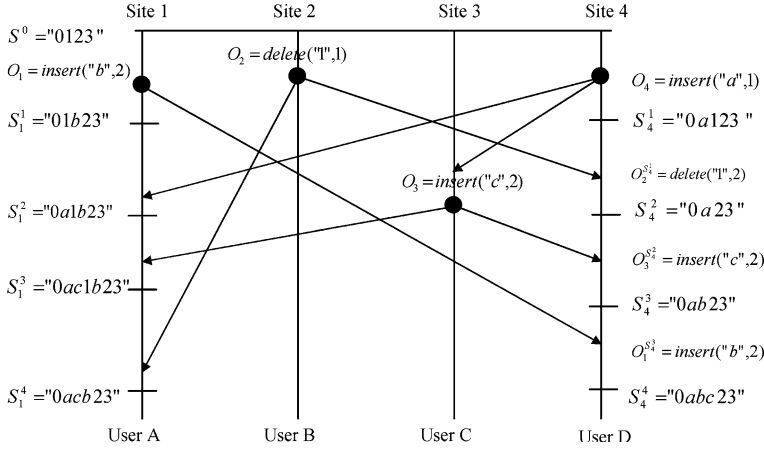


Figure 6. A divergence scenario of IMOR.

code to break the tie. As a result, we have $O_1^{S_4^3} = \text{insert}('b', 2)$. After it is executed, the state of site 4 becomes $S_4^4 = '0abc23'$. This result is not correct, because 'b' should be after 'c' in position due to the landmark character '1'.

Furthermore, consider site 1 in Figure 6. After O_1 is executed, we have $S_1^1 = '01b23'$. When O_4 arrives, $O_4^{S_1^1} = \text{IT}(O_4, O_1) = \text{insert}('a', 1)$, after the execution of which we get $S_1^2 = '0a1b23'$. Then O_3 arrives. The history buffer of site 1 is $[O_1^{S_1^0}, O_4^{S_1^1}]$. We have $(O_4 \rightarrow O_3) \parallel O_1$. It does not make sense to inclusively transform O_3 against O_1 (by directly using their IT functions) because their position parameters are relative to different states. This is a typical dOPT puzzle scenario. Although it is not clear how the control algorithm of IMOR works due to the absence of critical detail in Imine et al. (2003), it is fair to assume that IMOR is able to correctly solve the dOPT puzzle, e.g., if it inherits the GOTO-like control algorithm of SOCT2. The correct executable form of O_3 should be $O_3^{S_1^2} = \text{insert}('c', 2)$. After it is executed, we get $S_1^3 = '0ac1b23'$. Now O_2 arrives. It is concurrent to all operations currently in the history buffer of site 1. No matter how they are ordered, due to the IT functions of IMOR, we get $O_2^{S_1^3} = \text{delete}('1', 3)$. After it is executed, the final state of site 1 becomes $S_1^4 = '0acb23'$, which is obviously divergent from state $S_4^4 = '0abc23'$.

11.3. soct2

SOCT2 Suleiman et al. (1997), Suleiman et al. (1998) addresses the TP2 puzzle by introducing two new parameters to the insert operation $\text{insert}(c, p, b, a)$, where b is the set of concurrent operations that have deleted

characters before the inserted character, and a is the set of concurrent operations that have deleted characters after the inserted character. This unique way of defining the insert operation seems to show some promise in solving the TP2 problem. However, counterexamples identified in Imine et al. (2004), Oster et al. (2005) show that SOCT2 fails to achieve convergence in some cases.

The following two scenarios show that SOCT2 and SDT have different strengths in preserving user intentions. On one hand, consider the following multi-user game, in which two users concurrently edit a string such that characters are always in alphabetic order. Let the initial state be "135". User A inserts '4' between '3' and '5', yielding "1345". Concurrently user B deletes '3' and then inserts '2', yielding "125". Either result satisfies the game constraint. However, after these three operations are all executed, SOCT2 converges in state "1425", which violates the game constraint.

On the other hand, the following game scenario reveals that SOCT2 may be better than SDT in some cases. Suppose two users edit a shared string of digits such that all digits are always in ascending order. In SOCT2, when two different digits are inserted concurrently at the same position, its tie-breaking policy always puts the digit with a smaller ASCII value before the other. That is, if the two digits in question are '1' and '3', respectively, then the result will always be "13", which ensures the game constraint. However, SDT compares site ids to break the tie in this case, which may yield "31" if '1' is inserted by the user with a larger site id.

11.4. DISCUSSIONS

It has long been accepted in the literature that convergence must be somehow constrained in collaborative systems (Ellis and Gibbs, 1989; Ressel et al., 1996; Suleiman et al., 1998; Sun and Ellis, 1998; Sun et al., 1998; Sun, 2002b). The constraint of intention preservation (Suleiman et al., 1998; Sun et al., 1998) has been well established in this context. As an alternative constraint, our notion of operation effects relation is easier to formalize and verify. As a result, we can say something definitive about the correctness of OT algorithms for the first time.

This advantage of our work also implies its limitations. As a system level constraint, effects relation leaves out application semantics. For example, consider that two users concurrently modify the same word "book", one to make it "a book" while the other to make it "books". The result, "a books", will be acceptable under system level constraints such as this work and Suleiman et al. (1998), Sun et al. (1998) as presented. However, if the concept of user intention (Suleiman et al., 1998) or operation intention (Sun et al., 1998; Sun, 2002b) were interpreted as including domain knowledge,

operations could be transformed differently and the grammatically incorrect result “a books” could be identified as unacceptable.

Therefore, our constraint of effects relation complements rather than replaces the constraint of intention preservation. They represent different assumptions, philosophies, and tradeoffs and serve different purposes in approaching the same problem of consistency maintenance in collaborative systems. While under-formalization may make the problem of correctness verification less tractable, a conservative formalization may yield system states that are less intuitive to the end users. Nonetheless, we share the same long-term goal as Suleiman et al. (1998), Sun et al. (1998) in terms of preserving user or operation intentions.

12. Conclusions and future directions

This article presents an approach that ensures convergence in peer-to-peer real-time group editors, together with a formal proof of correctness. It is the first and the only work that is proved to achieve convergence in the presence of arbitrary communication and transformation paths, to the best of our knowledge. In addition, our CSM consistency model is the first work that formulates system-level conditions for proving the correctness of OT algorithms. This will help in our design and evaluation of next-generation group editors such that we can prove the correctness of syntactic consistency algorithms and do usability experiments on user interface features (e.g., Sun, 2002a, b; Sun et al., 2004) that are designed to help the users preserve semantic consistency and their intentions.

Due to space reasons, a more rigorous definition of the effects relation is left to Li and Li (2005c). We analyze the performance of SDT in Li and Li (2005a): its worst-case time complexity is $O(n^3)$ and the expected time is $O(n^2)$, where n is the size of the operation log. It is also formally proved in Li and Li (2005a) that the worst case only happens when there is a tie between the two insertions to be inclusively transformed. Our recent work in Li and Li (2005b) reveals an alternative approach, called Landmark-Based Transformation or LBT, to enforcing the effects relation. LBT addresses consistency problems at the control algorithm level without using extra operation parameters as in SOCT2 and this work.

We plan to investigate the following directions in future work: First, the idea of state difference transformation is only explored in this article to address the correctness or consistency problems. It can potentially improve the efficiency of OT algorithms due to its compressed representation of operations. For simplicity and space reasons, the presented algorithms focus on characterwise operations. We plan to extend our work to stringwise operations and tree based data structures as in Sun et al. (1998), Davis et al.

(2002). Mechanisms such as group undo (Sun, 2002b), optional locking (Sun, 2002a), and multi-version (Sun et al., 2004) can also be developed atop the concept of SDT.

Second, our notion of operation effects relation is only a system level constraint. Although it partially formalizes operation intentions, much is still left to be desired in terms of achieving semantic consistency and intention preservation. In future work, we plan to investigate more advanced techniques such that user or operation intentions can be somehow formalized and enforced in a collaborative environment. Recent progress in collaborate editing systems that attempts to maintain semantic consistency in the form of constraints, e.g., (Li and Patrao, 2001; Skaf-Molli et al., 2003; Lin et al., 2005) has cast a light in this direction.

Acknowledgments

We thank the anonymous expert referees for their valuable comments which helped improve the presentation of this paper. Especially we thank Chengzheng Sun for his valuable comments and discussions on an early draft. We are profoundly grateful to the Associate Editor, Prasun Dewan, for coordinating the review process. This research was supported in part by the National Science Foundation under CAREER award IIS-0133871.

Note

1. The approach of Imine et al. (2003) was not named. Here we name it as IMOR for convenience.

References

- Baecker, R.M., D. Nastos, I.R. Posner, and K.L. Mawby, (1993): The User-Centered Iterative Design of Collaborative Writing Software. In *Proceedings of the InterCHI'93 Conference on Human Factors in Computing Systems*, pp. 309–405.
- Bellini, P., P. Nesi and M.B. Spinu (2002): Cooperative Visual Manipulation of Music Notation. *ACM Transactions on Computer-Human Interaction*, vol. 9, no. 3 Sept., pp. 194–237.
- Bentley, R. and P. Dourish (1995a): Medium Versus Mechanism: Supporting Collaboration through Customisation. In *Proceedings of the European Conference on Computer-Supported Cooperative Work (ECSCW'95)*. Stockholm, Sweden.
- Bentley, R. and P. Dourish (1995b): Medium Versus Mechanism: Supporting Collaboration through Customization. In *ECSCW'95 Proceedings*.
- Davis, A.H., C. Sun and J. Lu (2002): Generalizing Operational Transformation to the Standard General Markup Language. In *ACM CSCW'02*, pp. 58–67.
- Dewan, P., R. Choudhary and H. Shen (1994): An Editing-based Characterization of the Design Space of Collaborative Applications. *Journal of Organizational Computing*, vol. 4, no. 3, pp. 219–240.

- Dourish, P. (1996): Consistency Guarantees: Exploiting Application Semantics for Consistency Management in a Collaboration Toolkit. In *ACM CSCW'96 Proceedings*. Cambridge, MA, pp. 268–277.
- Edwards, W.K. (1996): Policies and Roles in Collaborative Applications. In *ACM CSCW'96 Proceedings*, pp. 11–20.
- Ellis, C.A. and S.J. Gibbs (1989): Concurrency Control in Groupware Systems. In *Proceedings of the ACM SIGMOD'89 Conference on Management of Data*. Portland Oregon, pp. 399–407.
- Ellis, C.A., S.J. Gibbs and G.L. Rein (1991): Groupware: Some Issues and Experiences. *Communications of the ACM*, vol. 34, no. 1 Jan., pp. 38–58.
- Greenberg, S. and D. Marwood (1994): Real-time Groupware as a Distributed System: Concurrency Control and its Effect on the Interface. In *ACM CSCW'94 Proceedings*. Chapel Hill, NC, pp. 207–217.
- Grudin, J. (1994): Groupware and Social Dynamics: Eight Challenges for Developers. *Communications of the ACM*, vol. 37, no. 1, pp. 92–105.
- Gutwin, C. and S. Greenberg (2002): A Descriptive Framework of Workspace Awareness for Realtime Groupware. *Computer Supported Cooperative Work*, vol. 11, no. 1–2, pp. 411–446.
- Hymes, C.M. and G.M. Olson (1992): Unblocking Brainstorming through the Use of Simple Group Editor. In *ACM CSCW'92 Proceedings*, pp. 99–106.
- Imine, A., P. Molli, G. Oster and M. Rusinowitch (2003): Proving Correctness of Transformation Functions in Real-time Groupware. In *Proceedings of the European Conference on Computer Supported Cooperative Work (ECSCW'03)*.
- Imine, A., P. Molli, G. Oster and M. Rusinowitch (2004): Achieving Convergence with Operational Transformation in Distributed Groupware Systems. Technical Report 5188 (May), INRIA.
- Knister, M.J. and A. Prakash (1994): A Framework for Undoing Actions in Collaborative Systems. *ACM Transactions on Computer-Human Interaction*, vol. 1, no. 4 Dec., pp. 295–330.
- Li, D. and R. Li (2002): Transparent Sharing and Interoperation of Heterogeneous Single-user Applications. In *Proceedings of the ACM CSCW'02 Conference on Computer-Supported Cooperative Work*, pp. 246–255.
- Li, D. and R. Li (2005a): An Operational Transformation Algorithm and Performance Evaluation. *Computer-Supported Cooperative Work*. Accepted.
- Li, R. and D. Li (2005b): A Landmark-based Transformation Approach to Concurrency Control in Group Editors. In *Proceedings of the ACM GROUP'05 Conference on Supporting Group Work*. Sanibel Island, FL, pp. 284–293.
- Li, R. and D. Li (2005c): A New Operational Transformation Framework for Real-time Group Editors. *IEEE Transactions on Parallel and Distributed Systems*. Accepted.
- Li, R., D. Li and C. Sun (2004): A Time Interval based Consistency Control Algorithm for Interactive Groupware Applications. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, pp. 429–436.
- Li, D. and J. Patrao (2001): Demonstrational Customization of a Shared Whiteboard to Support User-Defined Semantic Relationships among Objects. In *Proceedings of the ACM SIGGROUP'01 Conference on Supporting Group Work*. Boulder, Colorado, pp. 97–106.
- Lin, K., D. Chen, C. Sun and G. Dromey (2005): Maintaining Constraints in Collaborative Graphic Systems: The CoGSE Approach. In *Proceedings of the European Conference on Computer-Supported Cooperative Work*.
- Neuwirth, C.M., D.S. Kaufer, R. Chandhok and J.H. Morris (1994): Computer Support for Distributed Collaborative Writing: Defining Parameters of Interaction. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work*, Chapel Hill, NC, pp. 145–152.

- Nichols, D.A., P. Curtis, M. Dixon and J. Lamping (1995): High-latency, Low-bandwidth Windowing in the Jupiter Collaboration System. In *ACM UIST'95 Proceedings*.
- Noel, S. and J.-M. Robert (2004): Empirical Study on Collaborative Writing: What do Co-authors do, Use, and Like. *Computer Supported Cooperative Work*, vol. 13, 63–89.
- Oster, G., P. Urso, P. Molli and A. Imine (2005): Real Time Group Editors without Operational Transformation. Technical Report 5580 (May), INRIA.
- Ressel, M., D. Nitsche-Ruhland and R. Gunzenhäuser (1996): An Integrating, Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors. In *Proceedings of the ACM CSCW'96 Conference on Computer-Supported Cooperative Work*, pp. 288–297.
- Shen, H. and C. Sun (2002): Flexible Notification for Collaborative Systems. In *Proceedings of the ACM CSCW'02 Conference on Computer-Supported Cooperative Work*, pp. 77–86.
- Shipman, F. and C. Marshall (1999): Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representation in Interactive Systems. *Computer Supported Cooperative Work*, vol. 8, no. 4, pp. 333–352.
- Skaf-Molli, H., P. Molli and G. Oster (2003): Semantic Consistency for Collaborative Systems. In *The 5th International Workshop on Collaborative Editing Systems, hosted by the ECSCW'03 Conference*. Helsinki.
- Suleiman, M., M. Cart and J. Ferrié (1997): Serialization of Concurrent Operations in a Distributed Collaborative Environment. In *ACM GROUP'97 Proceedings*. Phoenix, AZ.
- Suleiman, M., M. Cart and J. Ferrié (1998): Concurrent Operations in a Distributed and Mobile Collaborative Environment. In *Proceedings of the IEEE ICDE'98 International Conference on Data Engineering*, pp. 36–45.
- Sun, C. (2002a): Optional and Responsive Fine-grain Locking in Internet-based Collaborative Systems. *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 9 Sept., pp. 994–1008.
- Sun, C. (2002b): Undo as Concurrent Inverse in Group Editors. *ACM Transactions on Computer-Human Interaction*, vol. 9, no. 4 Dec., pp. 309–361.
- Sun, C. (2004): False-Tie Puzzle Solved in Reduce and Coword Group Editors. Personal Communication.
- Sun, C. and D. Chen (2002): Consistency Maintenance in Real-Time Collaborative Graphics Editing Systems. *ACM Transactions on Computer-Human Interaction*, vol. 9, no. 1 March, pp. 1–41.
- Sun, C. and C. Ellis (1998): Operational Transformation in Real-time Group Editors: Issues, Algorithms, and Achievements. In *Proceedings of the ACM CSCW'98 Conference on Computer-Supported Cooperative Work*, pp. 59–68.
- Sun, C., X. Jia, Y. Zhang, Y. Yang and D. Chen (1998): Achieving Convergence, Causality-Preservation, and Intention-Preservation in Real-Time Cooperative Editing Systems. *ACM Transactions on Computer-Human Interaction*, vol. 5, no. March, pp. 63–108.
- Sun, D., S. Xia, C. Sun and D. Chen (2004): Operational Transformation for Collaborative Word Processing. In *Proceedings of ACM CSCW'04 Conference on Computer-Supported Cooperative Work*, pp. 162–171.
- Vidot, N., M. Cart, J. Ferrie and M. Suleiman (2000): Copies Convergence in a Distributed Realtime Collaborative Environment. In *Proceedings of ACM CSCW'00 Conference on Computer-Supported Cooperative Work*, pp. 171–180.
- Xia, S. D. Sun, C. Sun, D. Chen and H. Shen (2004): Leveraging Single-user Applications for Multi-user Collaboration: The CoWord Approach. In *Proceedings of ACM CSCW'04 Conference on Computer-Supported Cooperative Work*, pp. 437–446.